2/11/2026 - 4:35:18 p.m.



2/11/2026 - 4:35:19 p.m.

Soy estudiante de la Maestría en Finanzas de EGADE (curso Productos Derivados y Administración de Riesgos Corporativos, Dr. Raymundo Díaz Robles). Mi equipo entrega la Tarea Clase 6: GARCH, Monte Carlo y árboles binomiales con NVDA, AMD y XOM, con cotizaciones del 30-oct-2026 y opciones que vencen el 18-dic-2026. Adjunto un PDF con los resultados reales de nuestro análisis en R. No escribas código todavía: resume en 8 líneas lo que entendiste del análisis y de los datos, y dime qué faltaría para que sea un análisis completo. Usa español y no uses guiones largos.

ChatGPT (GPT 5.2) •2/11/2026 - 4:35:19 p.m.

Plan:

1. Abrir el PDF adjunto y extraer su texto de forma automática.
2. Identificar qué activos, fechas, supuestos y métodos (GARCH, Monte Carlo, binomial) reporta el documento y qué salidas numéricas principales presenta.
3. Redactar un resumen en exactamente 8 líneas sobre datos y análisis.
4. Listar, de forma concisa, qué elementos faltan para considerar el análisis completo (supuestos, validaciones, métricas, comparación de modelos, etc.).

Analizaron NVDA, AMD y XOM con fecha spot 30-oct-2026 y opciones con vencimiento 18-dic-2026 (34 días naturales), comparando valuación y decisión de trading.
Estimaron volatilidad con GARCH(1,1) por acción (omega, alpha, beta), reportando persistencia, volatilidad de largo plazo, volatilidad “hoy” y vida media del shock.
Derivaron una curva de volatilidad GARCH esperada por horizonte (10, 30, 34, 126, 252 días) y la contrastaron con volatilidad reciente (30 días, 1 año y EWMA).
Valuaron calls y puts (strikes: NVDA 200, AMD 500, XOM 185) con árbol binomial CRR americano de 500 pasos usando σ histórica, σ GARCH y σ implícita, y lo compararon contra bid y ask de mercado.
Corrieron Monte Carlo con 100,000 trayectorias incorporando GARCH y mostraron que los precios MC son muy cercanos a los del árbol con σ GARCH.
Cuantificaron prima de ejercicio anticipado, prácticamente nula en calls y pequeña pero positiva en puts, y reportaron griegas para algunas posiciones.

Con una razón de “ventaja” (teórica vs mercado) concluyen sin ventaja clara de volatilidad en NVDA y AMD, y que XOM luce “cara” en volatilidad; proponen mantener solo apuestas direccionales (NVDA call, AMD put) y no comprar XOM por vol.
Para un análisis completo faltaría documentar supuestos clave (tasa libre de riesgo, dividendos, convención de días, si el subyacente paga dividendos esperados), calibración y diagnóstico del GARCH (datos usados, distribución de innovaciones, backtesting), y validación de mercado (cadena completa, elección de strike, smile, spreads, costos y sensibilidad a parámetros).

Thinking
code_execution_read_file

[DONE]

2/11/2026 - 4:36:23 p.m.

Gracias. Para aclarar los faltantes: la tasa libre de riesgo continua es 3.94% (la misma de la tarea de árboles). Los dividendos entran como rendimiento continuo: NVDA 0.47%, AMD 0% y XOM 2.25%. La volatilidad implícita viene de TradingView, que usa su propio modelo, por eso difiere un poco de la que sale de bid/ask. La posición de XOM (call 185 por 3 contratos) se mantiene en la tarea de Investopedia como tesis direccional, aunque la señal de volatilidad diga que la opción está cara. Con esto, continúa y dime qué faltaría ahora para un análisis completo. No uses guiones largos.



ChatGPT (GPT 5.2) •2/11/2026 - 4:36:24 p.m.

Con esas aclaraciones, lo principal que todavía faltaría para que el análisis sea “completo” (en el sentido de estar bien sustentado, reproducible y defendible) es:

1. Trazabilidad y limpieza de datos

- Definir exactamente la fuente y frecuencia de precios para el subyacente (close, adjusted close, o con dividendos) y el periodo usado para cada ventana (5, 3, 2 años; 1 año; 30 días).
- Tratamiento de días no hábiles, outliers, splits, y consistencia entre retornos usados en GARCH y los S0 reportados al 30-oct-2026.

2. Validación del modelo GARCH

- Diagnósticos formales: estacionariedad, autocorrelación de residuales y de residuales al cuadrado, pruebas ARCH, estabilidad de parámetros.

- Especificación de innovaciones: normal vs t-Student vs GED, y el impacto en colas (muy relevante para VaR y probabilidades).
- Backtesting out of sample: comparar pronósticos de varianza a 10 y 34 días contra realized volatility, y reportar errores (RMSE, QLIKE) o cobertura.

3. Coherencia entre GARCH, Monte Carlo y árbol

- Explicitar la dinámica simulada: si el MC usa GARCH bajo medida real P o se transformó a medida neutral al riesgo Q; y cómo se incorpora r y q en la simulación.
- En el árbol CRR, dejar claro si σ se toma constante (promedio a 34 días) o si se intenta incorporar término estructura de vol; hoy parece σ constante por escenario.

4. Calibración y comparación contra mercado de opciones

- Usar cadena más amplia: varios strikes y, idealmente, al menos dos vencimientos para ver smile y term structure, no solo un strike por activo.
- Extraer "vol implícita mid" desde bid y ask con su propio pricer (con r y q que ya definiste) para que la comparación sea consistente, y reportar el error de precio y/o error de IV.

5. Costos de transacción y ejecución

- Incorporar spreads, comisiones y slippage en el criterio de "ventaja" y en el P&L esperado; con spreads amplios, una ventaja pequeña se vuelve no operable.
- Precisar tamaño de contrato, multiplicador y si los costos totales ya consideran multiplicador 100 (parece que sí, pero conviene dejarlo explícito).

6. Métrica de decisión más sólida que la "razón"

- Definir formalmente la razón (qué divide entre qué) y justificar el umbral de señal.
- Complementar con un criterio económico: expected value y expected shortfall bajo el modelo, o utilidad, o probabilidad de pérdida condicionada, y sensibilidad a r, q, σ.

7. Gestión del riesgo de portafolio y escenario

- Agregar stress testing (saltos, gap risk, choque de volatilidad) y escenarios macro para NVDA, AMD, XOM.
- Para la tesis direccional de XOM, justificar el driver (evento, catalizador, horizonte) y medir cuánto del resultado depende de delta vs vega, ya que por vol "cara" el riesgo principal es la contracción de IV.

8. Reproducibilidad y documentación

- Dejar en el entregable los supuestos operativos completos: convención de días (ACT/365 o ACT/252), calendario, definición exacta de "10 días hábiles", semilla de simulación, y tablas con parámetros finales.

- Alinear todas las probabilidades reportadas (ITM, prob. de ganar, prob. de subir) con una única definición y medida (P o Q), porque hoy pueden mezclar interpretaciones.

Si me dices cómo definieron exactamente la "razón" y si las probabilidades (ganar, ITM) provienen del MC con GARCH o de otra aproximación, puedo señalar el ajuste mínimo necesario para que todo quede consistente medida real vs neutral al riesgo.

[DONE]

2/11/2026 - 4:36:40 p.m.

Vamos a construir en R, paso a paso, el análisis de la tarea Clase 6 (GARCH, Monte Carlo y árboles binomiales) para NVDA, AMD y XOM. Adjunto el CSV con los precios diarios hasta el 30-oct-2026. Escribe solo el primer bloque de R: lee el CSV, calcula rendimientos logarítmicos diarios en porcentaje para cada acción, y muestra una tabla con la volatilidad anualizada de los últimos 30 días y del último año. Usa solo R base, sin paquetes externos. Comenta el código en español y no uses guiones largos.

ChatGPT (GPT 5.2) •2/11/2026 - 4:36:40 p.m.

Plan:

1. Abrir el CSV adjunto y revisar nombres de columnas, formato de fechas y si los precios vienen en columnas separadas por ticker.
2. Asumir que hay una columna de fecha y columnas numéricas para NVDA, AMD y XOM; si no, detectar el formato real (por ejemplo, formato largo con columnas ticker y close).
3. Entregar el primer bloque en R base: lectura del CSV, ordenamiento por fecha, cálculo de rendimientos logarítmicos diarios en porcentaje por acción.
4. Calcular volatilidad anualizada de los últimos 30 días hábiles y del último año hábil (252 días), y mostrar una tabla con esos resultados.No veo el CSV en esta conversación todavía. Si lo adjuntas, lo leo y adapto el bloque al formato exacto (columnas y nombres). Mientras tanto, aquí tienes un primer bloque en R base que funciona si el CSV tiene una columna `Date` y tres columnas de precios `NVDA`, `AMD`, `XOM` (en caso contrario, dime los nombres reales de las columnas y lo ajusto).No veo el CSV adjunto en esta conversación. En cuanto lo subas, lo ajusto al formato exacto de tus columnas (por ejemplo `date`, `NVDA.Close`, etc.). Mientras tanto, este primer bloque en R base funciona si el archivo trae `Date` y las columnas de precios `NVDA`, `AMD`, `XOM`.

```
## Bloque 1: Lectura de datos, rendimientos logarítmicos y volatilidad (R
base)

# 1) Leer CSV (ajusta el nombre del archivo si es necesario)
datos <- read.csv("precios_clase6.csv", stringsAsFactors = FALSE)
```

```r
# 2) Convertir y ordenar por fecha
datos$Date <- as.Date(datos$Date)   # si tu fecha viene como "YYYY-MM-DD"
datos <- datos[order(datos$Date), ]

# 3) Extraer matriz de precios (numérica) en el orden requerido
precios <- datos[, c("NVDA", "AMD", "XOM")]
precios <- as.data.frame(lapply(precios, as.numeric))

# 4) Calcular rendimientos logarítmicos diarios en porcentaje
# r_t = 100 * ln(P_t / P_{t-1})
ret <- as.data.frame(lapply(precios, function(p) 100 * diff(log(p))))
ret$Date <- datos$Date[-1]  # fechas alineadas con rendimientos

# 5) Función de volatilidad anualizada usando 252 días hábiles
vol_anual <- function(x) {
  sd(x, na.rm = TRUE) * sqrt(252)
}

# 6) Volatilidad anualizada: últimos 30 días hábiles y último año (252
días)
n <- nrow(ret)
ult30 <- max(1, n - 30 + 1):n
ulty  <- max(1, n - 252 + 1):n

vol30 <- sapply(ret[, c("NVDA","AMD","XOM")], function(x)
vol_anual(x[ult30]))
vol1y <- sapply(ret[, c("NVDA","AMD","XOM")], function(x)
vol_anual(x[ulty]))

# 7) Tabla resumen
tabla_vol <- data.frame(
  Accion = c("NVDA", "AMD", "XOM"),
  Vol_30d_anual = as.numeric(vol30),
  Vol_1y_anual  = as.numeric(vol1y),
  row.names = NULL
)

print(tabla_vol)
```

Si tu CSV está en formato “largo” (columnas tipo `Date`, `Ticker`, `Close`) o si los nombres de columnas no son exactamente esos, pégame el encabezado (`names(datos)`) y la primera línea (`head(datos, 3)`) y lo adapto.

Thinking
[DONE]

2/11/2026 - 4:37:00 p.m.

> Siguiente bloque: estima un GARCH(1,1) por máxima verosimilitud para cada acción, con innovaciones normales, usando optim con restricciones (omega mayor que 0, alpha y beta mayores o iguales a 0, alpha más beta menor que 1). Muestra una tabla con omega, alpha, beta, persistencia, volatilidad de largo plazo anualizada, volatilidad de hoy anualizada y la vida media del shock en días.

ChatGPT (GPT 5.2) •2/11/2026 - 4:37:02 p.m.

```
## Bloque 2: Estimación GARCH(1,1) por MLE con innovaciones normales (R
base)
## Supone que ya existe el data.frame 'ret' del bloque anterior con
columnas NVDA, AMD, XOM
## Rendimientos en porcentaje diario

# Función: ajusta GARCH(1,1) a una serie x (en %), con media constante
removida
fit_garch11_norm <- function(x) {
  x <- as.numeric(x)
  x <- x[is.finite(x)]
  mu <- mean(x)
  e  <- x - mu
  n  <- length(e)

  # Varianza inicial para arrancar la recursión
  h0 <- var(e)
  if (!is.finite(h0) || h0 <= 0) h0 <- 1

  # Negativa de la log-verosimilitud (normal) con penalización por violar
alpha+beta<1
  nll <- function(par) {
    omega <- par[1]
    alpha <- par[2]
    beta  <- par[3]

    # Penalización fuerte si viola la condición de estacionariedad
    if (omega <= 0 || alpha < 0 || beta < 0) return(1e12)
    if (alpha + beta >= 0.999999) return(1e12 + 1e8 * (alpha + beta -
0.999999)^2)

    h <- numeric(n)
    h[1] <- h0

    for (t in 2:n) {
      h[t] <- omega + alpha * e[t - 1]^2 + beta * h[t - 1]
      if (!is.finite(h[t]) || h[t] <= 0) return(1e12)
    }

    ll <- -0.5 * (log(2 * pi) + log(h) + (e^2) / h)
    return(-sum(ll))
  }

  # Valores iniciales razonables (en unidades de %^2 por día)
  alpha0 <- 0.05
  beta0  <- 0.90
  omega0 <- max(1e-6, h0 * (1 - alpha0 - beta0))

  # Optimización con restricciones por cotas + penalización para
alpha+beta<1
  opt <- optim(
    par    = c(omega0, alpha0, beta0),
    fn     = nll,
```

```
    method = "L-BFGS-B",
    lower  = c(1e-12, 0, 0),
    upper  = c(Inf, 1, 1)
  )

  par <- opt$par
  omega <- par[1]; alpha <- par[2]; beta <- par[3]
  pers  <- alpha + beta

  # Recalcular h_t con parámetros finales para obtener "volatilidad de hoy"
  e <- e
  h <- numeric(n)
  h[1] <- h0
  for (t in 2:n) h[t] <- omega + alpha * e[t - 1]^2 + beta * h[t - 1]

  # Varianza de largo plazo y volatilidades anualizadas (sqrt(252))
  h_lr <- omega / (1 - pers)
  vol_lr_ann  <- sqrt(h_lr) * sqrt(252)
  vol_today_ann <- sqrt(h[n]) * sqrt(252)

  # Vida media del shock (en días hábiles)
  half_life <- if (pers > 0 && pers < 1) log(0.5) / log(pers) else NA_real_

  list(
    omega = omega, alpha = alpha, beta = beta,
    persistencia = pers,
    vol_lr_ann = vol_lr_ann,
    vol_today_ann = vol_today_ann,
    half_life = half_life,
    converged = (opt$convergence == 0),
    value = opt$value
  )
}

# Ajustar para cada acción
tickers <- c("NVDA", "AMD", "XOM")
ajustes <- lapply(tickers, function(tk) fit_garch11_norm(ret[[tk]]))
names(ajustes) <- tickers

# Armar tabla de resultados
tabla_garch <- data.frame(
  Accion = tickers,
  omega = sapply(ajustes, `[[`, "omega"),
  alpha = sapply(ajustes, `[[`, "alpha"),
  beta  = sapply(ajustes, `[[`, "beta"),
  Persistencia = sapply(ajustes, `[[`, "persistencia"),
  Vol_largo_plazo_anual = sapply(ajustes, `[[`, "vol_lr_ann"),
  Vol_hoy_anual = sapply(ajustes, `[[`, "vol_today_ann"),
  Vida_media_shock_dias = sapply(ajustes, `[[`, "half_life"),
  Convergio = sapply(ajustes, `[[`, "converged"),
  row.names = NULL
)

print(tabla_garch)
```

Si tus rendimientos no están en porcentaje sino en decimales, avísame para ajustar la escala (omega y las volatilidades cambian por un factor 100).

Thinking

[DONE]

2/11/2026 - 4:37:34 p.m.

Siguiente bloque: calcula la volatilidad EWMA de hoy con lambda 0.94 y el pronóstico de varianza GARCH a 10, 30, 34, 126 y 252 días con la fórmula de reversión a la media. Dame la volatilidad promedio esperada hasta el vencimiento (18-dic-2026, 34 días naturales) y compárala con la volatilidad implícita de TradingView mediante la razón σ GARCH / σ implícita, con umbral de 10%.

ChatGPT (GPT 5.2) •2/11/2026 - 4:37:34 p.m.

```
## Bloque 3: EWMA (lambda 0.94) y pronóstico de varianza GARCH por
reversión a la media
## Supone que ya corriste los Bloques 1 y 2 y tienes:
## - ret: data.frame con rendimientos diarios en % y columnas NVDA, AMD,
XOM
## - ajustes: lista con parámetros GARCH por ticker (omega, alpha, beta)
## Nota: todo en unidades de % (volatilidades en % anual)

# Tickers a procesar
tickers <- c("NVDA", "AMD", "XOM")

# 1) EWMA: volatilidad de hoy (annualizada) con lambda = 0.94
lambda <- 0.94

ewma_vol_hoy <- function(x, lambda = 0.94) {
  x <- as.numeric(x)
  x <- x[is.finite(x)]
  n <- length(x)
  if (n < 2) return(NA_real_)

  # Rendimientos centrados (opcional, ayuda a estabilidad)
  e <- x - mean(x)

  # Inicializar con varianza muestral
  s2 <- var(e)
  if (!is.finite(s2) || s2 <= 0) s2 <- 1

  # Recursión EWMA: s2_t = lambda*s2_{t-1} + (1-lambda)*e_{t-1}^2
  for (t in 2:n) {
    s2 <- lambda * s2 + (1 - lambda) * e[t - 1]^2
  }

  # Volatilidad anualizada en %
  sqrt(s2) * sqrt(252)
}

vol_ewma_hoy <- sapply(tickers, function(tk) ewma_vol_hoy(ret[[tk]],
lambda))
```

```r
# 2) Pronóstico de varianza GARCH a h días (en días hábiles) por reversión
a la media
# E[h {t+h}] = h bar + (alpha+beta)^h * (h t - h bar)
# donde h_bar = omega / (1 - alpha - beta)
# y h_t es la varianza condicional "hoy" (en %^2 por día), consistente con
el Bloque 2

garch_forecast_var <- function(x, fit, horizons = c(10, 30, 34, 126, 252))
{
  x <- as.numeric(x)
  x <- x[is.finite(x)]
  mu <- mean(x)
  e  <- x - mu
  n  <- length(e)

  omega <- fit$omega
  alpha <- fit$alpha
  beta  <- fit$beta
  pers  <- alpha + beta

  # Varianza inicial y varianza condicional de hoy (reconstruida)
  h0 <- var(e)
  if (!is.finite(h0) || h0 <= 0) h0 <- 1

  h <- numeric(n)
  h[1] <- h0
  for (t in 2:n) h[t] <- omega + alpha * e[t - 1]^2 + beta * h[t - 1]
  h_today <- h[n]

  # Varianza de largo plazo
  h_bar <- omega / (1 - pers)

  # Pronóstico de varianza por horizonte
  fvar <- sapply(horizons, function(H) {
    h_bar + (pers^H) * (h_today - h_bar)
  })

  list(
    h_today = h_today,
    h_bar = h_bar,
    pers = pers,
    fvar = fvar
  )
}

horizons <- c(10, 30, 34, 126, 252)

# Matrices para almacenar resultados
vol_garch_h <- matrix(NA_real_, nrow = length(tickers), ncol =
length(horizons),
                      dimnames = list(tickers, paste0(horizons, "d")))
vol_garch_prom_34d <- setNames(rep(NA_real_, length(tickers)), tickers)

for (tk in tickers) {
  fc <- garch_forecast_var(ret[[tk]], ajustes[[tk]], horizons)

  # Volatilidades anualizadas por horizonte (sqrt(252))
  vol_garch_h[tk, ] <- sqrt(fc$fvar) * sqrt(252)

  # 3) Volatilidad promedio esperada hasta el vencimiento
```

```
  # Tomamos el promedio simple de las varianzas pronosticadas para los
próximos T días
  # y anualizamos. Para 34 días naturales, usamos T = 34 (como en su
entrega).
  T <- 34
  fvar_path <- fc$h_bar + (fc$pers^(1:T)) * (fc$h_today - fc$h_bar)
  vol_garch_prom_34d[tk] <- sqrt(mean(fvar_path)) * sqrt(252)
}

# 4) Comparación contra volatilidad implícita (TradingView)
# Ajusta estos valores a los que tengas de TradingView (en % anual, no en
decimal).
# Si vienen como decimales (0.152), multiplícalos por 100.
sigma_impl_tv <- c(NVDA = NA_real_, AMD = NA_real_, XOM = NA_real_)

# Razón y señal con umbral 10%
razon <- vol_garch_prom_34d / sigma_impl_tv

senal <- ifelse(is.na(razon), NA_character_,
                ifelse(razon >= 1.10, "GARCH > implícita (posible barata en
vol)",
                ifelse(razon <= 0.90, "GARCH < implícita (posible cara en
vol)",
                                      "Sin ventaja clara (|diff| < 10%)")))

# 5) Tablas de salida
tabla_ewma <- data.frame(
  Accion = tickers,
  Vol_EWMA_hoy_anual = as.numeric(vol_ewma_hoy),
  row.names = NULL
)

tabla_garch_plazos <- data.frame(
  Accion = tickers,
  Vol_GARCH_10d  = as.numeric(vol_garch_h[, "10d"]),
  Vol_GARCH_30d  = as.numeric(vol_garch_h[, "30d"]),
  Vol_GARCH_34d  = as.numeric(vol_garch_h[, "34d"]),
  Vol_GARCH_126d = as.numeric(vol_garch_h[, "126d"]),
  Vol_GARCH_252d = as.numeric(vol_garch_h[, "252d"]),
  Vol_prom_esperada_34d = as.numeric(vol_garch_prom_34d),
  Sigma_implicita_TV = as.numeric(sigma_impl_tv),
  Razon_GARCH_sobre_Impl = as.numeric(razon),
  Senal_10pct = as.character(senal),
  row.names = NULL
)

print(tabla_ewma)
print(tabla_garch_plazos)
```

Para que la comparación quede lista, solo dime si tus volatilidades implícitas de TradingView están en porcentaje (por ejemplo 15.2) o en decimal (0.152), y pégame los tres valores para NVDA, AMD y XOM; los coloco en `sigma_impl_tv` y te queda la tabla final con la razón y la señal.

[DONE]

2/11/2026 - 4:37:54 p.m.

Siguiente bloque: simula 100,000 trayectorias de precio con GARCH(1,1) por acción hasta el vencimiento, y entrega una tabla con precio de hoy, percentiles 5, 25, 50, 75 y 95, probabilidad de subir y VaR al 99% a 10 días. Fija la semilla para que sea reproducible.

ChatGPT (GPT 5.2) •2/11/2026 - 4:37:54 p.m.

```
## Bloque 4: Monte Carlo con GARCH(1,1) hasta vencimiento (100,000
trayectorias)
## Supone que ya tienes:
## - datos: data.frame con Date y precios NVDA, AMD, XOM (del Bloque 1)
## - ret: data.frame con rendimientos en % y columnas NVDA, AMD, XOM (del
Bloque 1)
## - ajustes: lista con parámetros GARCH por ticker (del Bloque 2)

set.seed(123)                  # Semilla para reproducibilidad
N <- 100000                    # Número de trayectorias
T_venc <- 34                   # Pasos hasta vencimiento (34 días naturales,
como en la tarea)
T_var <- 10                    # Horizonte para VaR (10 días)

tickers <- c("NVDA", "AMD", "XOM")

# Precios de hoy (última fila del CSV)
S0 <- as.numeric(datos[nrow(datos), tickers])
names(S0) <- tickers

# Función para obtener h_t (varianza condicional de hoy) consistente con el
ajuste GARCH
garch_htoday <- function(x, fit) {
  x <- as.numeric(x)
  x <- x[is.finite(x)]
  mu <- mean(x)
  e  <- x - mu
  n  <- length(e)

  omega <- fit$omega
  alpha <- fit$alpha
  beta  <- fit$beta

  h0 <- var(e)
  if (!is.finite(h0) || h0 <= 0) h0 <- 1

  h <- numeric(n)
  h[1] <- h0
  for (t in 2:n) h[t] <- omega + alpha * e[t - 1]^2 + beta * h[t - 1]

  h[n]
}

# Función de simulación: devuelve ST, S10 y métricas de distribución
simula_precio_garch <- function(S0, x_ret_pct, fit, N, T_venc, T_var) {
  x_ret_pct <- as.numeric(x_ret_pct)
  x_ret_pct <- x_ret_pct[is.finite(x_ret_pct)]
```

```
  # Drift (media histórica diaria) en % para la simulación
  mu <- mean(x_ret_pct)

  omega <- fit$omega
  alpha <- fit$alpha
  beta  <- fit$beta

  # Inicializar precio y varianza condicional de hoy
  S <- rep(S0, N)
  h <- rep(garch_htoday(x_ret_pct, fit), N)

  S10 <- rep(NA_real_, N)

  for (t in 1:T_venc) {
    z <- rnorm(N)                        # innovaciones normales
    e <- sqrt(h) * z                     # shock en % (innovación)
    r <- mu + e                          # rendimiento diario simulado en %
    S <- S * exp(r / 100)                # actualizar precio

    # Actualizar varianza condicional para el siguiente día
    h <- omega + alpha * (e^2) + beta * h

    if (t == T_var) S10 <- S
  }

  # Percentiles del precio a vencimiento
  p <- as.numeric(quantile(S, probs = c(0.05, 0.25, 0.50, 0.75, 0.95),
na.rm = TRUE))
  names(p) <- c("P5","P25","P50","P75","P95")

  # Probabilidad de subir a vencimiento
  prob_subir <- mean(S > S0)

  # VaR 99% a 10 días: pérdida relativa (S0 - S10)/S0
  loss10 <- (S0 - S10) / S0
  var99 10d <- as.numeric(quantile(loss10, probs = 0.99, na.rm = TRUE))

  list(
    P5 = p["P5"], P25 = p["P25"], P50 = p["P50"], P75 = p["P75"], P95 =
p["P95"],
    prob subir = prob subir,
    var99_10d = var99_10d
  )
}

# Ejecutar simulación por acción y armar tabla final
res <- lapply(tickers, function(tk) {
  simula_precio_garch(
    S0 = S0[tk],
    x ret pct = ret[[tk]],
    fit = ajustes[[tk]],
    N = N,
    T_venc = T_venc,
    T_var = T_var
  )
})
names(res) <- tickers

tabla_mc <- data.frame(
  Accion = tickers,
```

```
  Precio_hoy = as.numeric(S0[tickers]),
  P5  = sapply(res, `[[`, "P5"),
  P25 = sapply(res, `[[`, "P25"),
  P50 = sapply(res, `[[`, "P50"),
  P75 = sapply(res, `[[`, "P75"),
  P95 = sapply(res, `[[`, "P95"),
  Prob_de_subir = sapply(res, `[[`, "prob_subir"),
  VaR 99pct 10d = sapply(res, `[[`, "var99 10d"),
  row.names = NULL
)

print(tabla_mc)
```

Si quieres que el Monte Carlo sea bajo medida neutral al riesgo para valuar opciones, en el siguiente bloque cambiamos el drift a `r - q - 0.5*h` (en log) y usamos tus tasas continuas por acción. Aquí lo dejé como simulación "real" basada en la media histórica, que es lo típico para percentiles y VaR.

Thinking
[DONE]

2/11/2026 - 4:38:22 p.m.

Siguiente bloque: implementa un árbol binomial CRR americano de 500 pasos con rendimiento por dividendo continuo q, para calls y puts. Usa tasa libre de riesgo continua de 3.94%, y q de 0.47% para NVDA, 0% para AMD y 2.25% para XOM. Valúa NVDA call 200, AMD put 500 y XOM call 185 con σ histórica, σ GARCH y σ implícita, y compara contra bid y ask. Agrega la prima de ejercicio anticipado (americana menos europea).



ChatGPT (GPT 5.2) •2/11/2026 - 4:38:22 p.m.

```
## Bloque 5: Árbol binomial CRR (americano y europeo) con dividend yield
continuo q
## Requiere que ya tengas:
## - datos: data.frame con Date y precios NVDA, AMD, XOM (última fila es
30-oct-2026)
## - ret: rendimientos diarios en % para NVDA, AMD, XOM
## - vol garch prom 34d: volatilidad GARCH promedio esperada a 34 días (del
Bloque 3), en % anual
## También necesitas capturar tus volatilidades implícitas de TradingView y
bid/ask del mercado.

# Parámetros de mercado
r <- 0.0394                    # tasa libre de riesgo continua
T <- 34/365                    # 34 días naturales a años (ACT/365)
Nsteps <- 500                  # pasos del árbol

q yield <- c(NVDA = 0.0047, AMD = 0.0000, XOM = 0.0225)

# Opciones a valuar
```

```r
inst <- data.frame(
  Accion = c("NVDA", "AMD", "XOM"),
  Tipo   = c("call", "put", "call"),
  K      = c(200, 500, 185),
  stringsAsFactors = FALSE
)

# Precio spot de hoy
S0 <- as.numeric(datos[nrow(datos), inst$Accion])
names(S0) <- inst$Accion

# Volatilidad histórica anual (último año) desde rendimientos del Bloque 1
# ret está en % diario; anualizamos: sd * sqrt(252)
vol_hist_1y <- sapply(inst$Accion, function(tk) sd(ret[[tk]], na.rm = TRUE)
* sqrt(252)) # en %

# Volatilidad GARCH promedio esperada a 34 días (en % anual)
# Se asume creada en el Bloque 3 con nombres NVDA, AMD, XOM
vol_garch_34 <- vol_garch_prom_34d[inst$Accion]  # en %

# Volatilidad implícita TradingView (captúrala aquí; en % anual)
vol_impl_tv <- c(NVDA = NA_real_, AMD = NA_real_, XOM = NA_real_)

# Bid y Ask de mercado (USD por acción) para las tres opciones
bid_mkt <- c(NVDA = NA_real_, AMD = NA_real_, XOM = NA_real_)
ask_mkt <- c(NVDA = NA_real_, AMD = NA_real_, XOM = NA_real_)

# Implementación CRR con valuación europea y americana
crr_price <- function(S0, K, r, q, sigma_ann, T, Nsteps, type =
c("call","put"), american = TRUE) {
  type <- match.arg(type)

  # sigma anual viene en %; convertir a decimal
  sigma <- sigma_ann / 100

  dt <- T / Nsteps
  u <- exp(sigma * sqrt(dt))
  d <- 1 / u

  disc <- exp(-r * dt)

  # Probabilidad neutral al riesgo con dividend yield continuo q
  p <- (exp((r - q) * dt) - d) / (u - d)

  # Guardas básicas para estabilidad numérica
  if (!is.finite(p) || p < 0 || p > 1) {
    return(NA_real_)
  }

  # Precios terminales del subyacente: S T(j) = S0 * u^j * d^(N-j), j =
0..N
  j <- 0:Nsteps
  ST <- S0 * (u^j) * (d^(Nsteps - j))

  # Payoff terminal
  if (type == "call") {
    V <- pmax(ST - K, 0)
  } else {
    V <- pmax(K - ST, 0)
  }
```

```
  # Backward induction
  for (i in (Nsteps - 1):0) {
    # Valores esperados descontados
    V <- disc * (p * V[2:(i + 2)] + (1 - p) * V[1:(i + 1)])

    if (american) {
      # Precio del subyacente en el nodo (i, j), j = 0..i
      j <- 0:i
      Sij <- S0 * (u^j) * (d^(i - j))

      # Valor intrínseco
      if (type == "call") {
        intrinsic <- pmax(Sij - K, 0)
      } else {
        intrinsic <- pmax(K - Sij, 0)
      }

      V <- pmax(V, intrinsic)
    }
  }

  V[1]
}

# Función para valuar un instrumento con tres sigmas y devolver tabla
valua_instrumento <- function(tk, tipo, K, r, q, T, Nsteps,
                              sigma_hist, sigma_garch, sigma_impl, bid,
ask) {

  # Árbol europeo y americano para cada sigma
  sigmas <- c(Historica_1y = sigma_hist, GARCH_34d = sigma_garch,
Implicita_TV = sigma_impl)

  out <- lapply(names(sigmas), function(nm) {
    sig <- sigmas[[nm]]

    price_eur <- crr_price(S0 = S0[tk], K = K, r = r, q = q, sigma_ann =
sig,
                           T = T, Nsteps = Nsteps, type = tipo, american =
FALSE)
    price_ame <- crr_price(S0 = S0[tk], K = K, r = r, q = q, sigma_ann =
sig,
                           T = T, Nsteps = Nsteps, type = tipo, american =
TRUE)

    data.frame(
      Accion = tk,
      Tipo = tipo,
      K = K,
      Sigma fuente = nm,
      Sigma_anual_pct = sig,
      Precio_Europeo = price_eur,
      Precio_Americano = price_ame,
      Prima ejercicio anticipado = price ame - price eur,
      Bid = bid,
      Ask = ask,
      row.names = NULL,
      stringsAsFactors = FALSE
    )
```

```
  })

  do.call(rbind, out)
}

# Construir tabla final para NVDA call 200, AMD put 500, XOM call 185
tabla_arbol <- do.call(rbind, lapply(1:nrow(inst), function(i) {
  tk <- inst$Accion[i]
  valua_instrumento(
    tk = tk,
    tipo = inst$Tipo[i],
    K = inst$K[i],
    r = r,
    q = q_yield[tk],
    T = T,
    Nsteps = Nsteps,
    sigma_hist = vol_hist_1y[tk],
    sigma_garch = vol_garch_34[tk],
    sigma_impl = vol_impl_tv[tk],
    bid = bid_mkt[tk],
    ask = ask_mkt[tk]
  )
}))

print(tabla_arbol)
```

Para que la comparación contra bid y ask quede completa, llena estos seis valores en el bloque: `vol_impl_tv` (NVDA, AMD, XOM) y `bid_mkt`, `ask_mkt` para cada opción. Si tus bid y ask vienen de una captura con cifras por acción, así está bien; si vienen por contrato, divide entre 100 antes de pegarlos.

[DONE]

2/11/2026 - 4:38:44 p.m.

Siguiente bloque: calcula delta, gamma y theta desde los nodos del árbol, y vega y rho por diferencias finitas, para las posiciones NVDA call 200 (2 contratos), AMD put 500 (1) y XOM call 185 (3). Muestra también los nodos de los primeros 2 pasos y del paso 4 con la decisión ejercer o esperar.



ChatGPT (GPT 5.2) •2/11/2026 - 4:38:45 p.m.

```
## Bloque 6: Griegas desde el árbol CRR y nodos con decisión ejercer o
esperar (R base)
## Supone que ya tienes del Bloque 5:
## - r, T, Nsteps, q_yield, inst, S0
## - vol_impl_tv (en % anual) para NVDA, AMD, XOM
## Nota: aquí calculo griegas del precio americano usando sigma implícita
(TradingView).
## Si prefieres sigma GARCH o histórica, cambia sigma_use.
```

```r
# Posiciones (contratos). Multiplicador estándar: 100 acciones por contrato
pos <- data.frame(
  Accion = c("NVDA", "AMD", "XOM"),
  Tipo   = c("call", "put", "call"),
  K      = c(200, 500, 185),
  Contratos = c(2, 1, 3),
  stringsAsFactors = FALSE
)
mult <- 100

# Bumps para diferencias finitas
bump_sigma_pto <- 1.0   # 1 punto de volatilidad (por ejemplo 15% a 16%)
bump r pto     <- 0.01  # 1 punto porcentual de tasa (por ejemplo 3.94% a
4.94%)

# Función: precio americano CRR y captura de nodos para i = 1, 2 y 4
crr_american_with_nodes <- function(S0, K, r, q, sigma_ann, T, Nsteps, type
= c("call","put"),
                                    keep_steps = c(1,2,4)) {
  type <- match.arg(type)

  sigma <- sigma_ann / 100
  dt <- T / Nsteps
  u <- exp(sigma * sqrt(dt))
  d <- 1 / u
  disc <- exp(-r * dt)
  p <- (exp((r - q) * dt) - d) / (u - d)

  if (!is.finite(p) || p < 0 || p > 1) {
    return(list(price = NA_real_, nodes = NULL, dt = dt))
  }

  # Terminal
  j <- 0:Nsteps
  ST <- S0 * (u^j) * (d^(Nsteps - j))
  if (type == "call") V <- pmax(ST - K, 0) else V <- pmax(K - ST, 0)

  nodes <- list()

  # Backward induction
  for (i in (Nsteps - 1):0) {
    # Continuación (esperar)
    V_cont <- disc * (p * V[2:(i + 2)] + (1 - p) * V[1:(i + 1)])

    # Intrínseco (ejercer)
    j <- 0:i
    Sij <- S0 * (u^j) * (d^(i - j))
    if (type == "call") intrinsic <- pmax(Sij - K, 0) else intrinsic <-
pmax(K - Sij, 0)

    # Americano
    exercise <- intrinsic > V_cont
    V <- pmax(V_cont, intrinsic)

    # Guardar nodos solicitados
    if (i %in% keep steps) {
      nodes[[paste0("step_", i)]] <- data.frame(
        step = i,
        j = j,
        S = Sij,
```

```
        Continuar = V_cont,
        Intrinseco = intrinsic,
        Valor americano = V,
        Decision = ifelse(exercise, "EJERCER", "ESPERAR"),
        row.names = NULL
      )
    }
  }

  list(price = V[1], nodes = nodes, dt = dt, u = u, d = d, p = p)
}

# Función: griegas del árbol (delta, gamma, theta de nodos; vega y rho por
diferencias finitas)
crr_greeks <- function(S0, K, r, q, sigma_ann, T, Nsteps, type =
c("call","put")) {
  type <- match.arg(type)

  # Correr una vez y guardar nodos de pasos 1 y 2
  out <- crr_american_with_nodes(S0, K, r, q, sigma_ann, T, Nsteps, type,
keep_steps = c(1,2))
  if (!is.finite(out$price)) {
    return(list(delta = NA, gamma = NA, theta = NA, vega = NA, rho = NA,
nodes12 = NULL))
  }

  dt <- out$dt

  n1 <- out$nodes[["step_1"]]
  n2 <- out$nodes[["step_2"]]

  # Paso 1: dos nodos (down j=0, up j=1)
  Vd <- n1$Valor_americano[n1$j == 0]
  Vu <- n1$Valor_americano[n1$j == 1]
  Sd <- n1$S[n1$j == 0]
  Su <- n1$S[n1$j == 1]

  delta <- (Vu - Vd) / (Su - Sd)

  # Paso 2: tres nodos (dd j=0, ud j=1, uu j=2)
  Vdd <- n2$Valor americano[n2$j == 0]
  Vud <- n2$Valor_americano[n2$j == 1]
  Vuu <- n2$Valor_americano[n2$j == 2]

  Sdd <- n2$S[n2$j == 0]
  Sud <- n2$S[n2$j == 1]
  Suu <- n2$S[n2$j == 2]

  # Deltas en nodos intermedios para gamma
  delta up   <- (Vuu - Vud) / (Suu - Sud)
  delta down <- (Vud - Vdd) / (Sud - Sdd)

  gamma <- (delta_up - delta_down) / ((Suu - Sdd) / 2)

  # Theta: aproximación con el nodo medio del paso 2 (ud), en USD por día
  # theta ≈ (V {t+2dt, mid} - V {t}) / (2dt), y luego /365 para llevar a
"por día"
  V0 <- out$price
  theta_per_year <- (Vud - V0) / (2 * dt)
  theta_per_day <- theta_per_year / 365
```

```
  # Vega por diferencias finitas: 1 punto de vol en sigma anual (en %)
  price_up <- crr_american_with_nodes(S0, K, r, q, sigma_ann +
bump_sigma_pto, T, Nsteps, type,
                                      keep_steps = integer(0))$price
  price_dn <- crr_american_with_nodes(S0, K, r, q, sigma_ann -
bump_sigma_pto, T, Nsteps, type,
                                      keep_steps = integer(0))$price
  vega <- (price_up - price_dn) / (2 * bump_sigma_pto)  # USD por +1 punto
de vol

  # Rho por diferencias finitas: 1 punto porcentual en r (0.01)
  price_ru <- crr_american_with_nodes(S0, K, r + bump_r_pto, q, sigma_ann,
T, Nsteps, type,
                                      keep_steps = integer(0))$price
  price_rd <- crr_american_with_nodes(S0, K, r - bump_r_pto, q, sigma_ann,
T, Nsteps, type,
                                      keep_steps = integer(0))$price
  rho <- (price_ru - price_rd) / (2 * bump_r_pto)       # USD por +1 punto
de tasa

  list(delta = delta, gamma = gamma, theta = theta_per_day, vega = vega,
rho = rho,
       nodes12 = out$nodes)
}

# Calcular griegas por posición usando sigma implícita TradingView
res_greeks <- lapply(1:nrow(pos), function(i) {
  tk <- pos$Accion[i]
  tipo <- pos$Tipo[i]
  K <- pos$K[i]
  contratos <- pos$Contratos[i]

  sigma_use <- vol_impl_tv[tk]  # cambiar a vol_garch_34[tk] o
vol_hist_1y[tk] si lo deseas

  g <- crr_greeks(S0 = S0[tk], K = K, r = r, q = q_yield[tk],
                  sigma_ann = sigma_use, T = T, Nsteps = Nsteps, type =
tipo)

  data.frame(
    Accion = tk,
    Tipo = tipo,
    K = K,
    Sigma_usada_pct = sigma_use,
    Contratos = contratos,
    Delta_opcion = g$delta,
    Gamma_opcion = g$gamma,
    Theta_opcion_USD_dia = g$theta,
    Vega_opcion_USD_por_1pto = g$vega,
    Rho_opcion_USD_por_1pto = g$rho,
    Delta_posicion_acciones = g$delta * contratos * mult,
    Gamma_posicion = g$gamma * contratos * mult,
    Theta_posicion_USD_dia = g$theta * contratos * mult,
    Vega_posicion_USD_por_1pto = g$vega * contratos * mult,
    Rho_posicion_USD_por_1pto = g$rho * contratos * mult,
    row.names = NULL
  )
})
```

```
tabla_greeks <- do.call(rbind, res_greeks)
print(tabla_greeks)

# Nodos de los primeros 2 pasos y del paso 4 con decisión ejercer o esperar
# Se muestran para cada instrumento con sigma implícita
for (i in 1:nrow(pos)) {
  tk <- pos$Accion[i]
  tipo <- pos$Tipo[i]
  K <- pos$K[i]
  sigma_use <- vol_impl_tv[tk]

  cat("\n========================\n")
  cat("Nodos y decisión:", tk, tipo, "K =", K, "sigma =", sigma_use, "%\n")
  cat("========================\n")

  dbg <- crr_american_with_nodes(S0 = S0[tk], K = K, r = r, q =
q_yield[tk],
                                 sigma_ann = sigma_use, T = T, Nsteps =
Nsteps,
                                 type = tipo, keep_steps = c(1,2,4))

  print(dbg$nodes[["step_1"]])
  print(dbg$nodes[["step_2"]])
  print(dbg$nodes[["step_4"]])
}
```

Si me confirmas que tus `vol_impl_tv` ya están capturadas (en % anual) y que el multiplicador es 100, esto te debe reproducir deltas y griegas a nivel posición. Si tus griegas objetivo son "por contrato" en vez de "por posición", se dejan solo las columnas `*_opcion`.

[DONE]

2/11/2026 - 4:39:53 p.m.

Último bloque: calcula la señal de volatilidad (razón σ GARCH / σ implícita, umbral 10%), el costo y los puntos de equilibrio del straddle, y la rentabilidad a 2 semanas: simula 10 días hábiles, revalúa cada opción con el árbol y reporta P&L esperado, probabilidad de ganar y percentiles 5 y 95 por posición.



ChatGPT (GPT 5.2) •2/11/2026 - 4:39:54 p.m.

```
## Bloque 7: Señal de volatilidad, straddle y rentabilidad a 2 semanas (10
días hábiles)
## Supone que ya tienes de bloques previos:
## - datos, ret
## - ajustes (GARCH por acción)
## - vol_garch_prom_34d (en % anual) y vol_impl_tv (en % anual)
## - r, q_yield
## - bid mkt, ask mkt (para las opciones de interés)
## - Funciones: crr_price() del Bloque 5, garch_htoday() del Bloque 4
```

```r
tickers <- c("NVDA", "AMD", "XOM")

# Horizonte y vencimiento
T_total_days <- 34
T_total <- T_total_days / 365

T_2w_days <- 10            # 10 días hábiles para revaluación
T rem days <- T total days - T 2w days
T_rem <- T_rem_days / 365

Npaths <- 100000
set.seed(123)

# Spots de hoy
S0 <- as.numeric(datos[nrow(datos), tickers])
names(S0) <- tickers

# 1) Señal de volatilidad: razón sigma GARCH / sigma implícita, umbral 10%
razon_vol <- vol_garch_prom_34d[tickers] / vol_impl_tv[tickers]

senal_vol <- ifelse(is.na(razon_vol), NA_character_,
                    ifelse(razon_vol >= 1.10, "POSIBLE BARATA en vol (GARCH
> implícita)",
                    ifelse(razon_vol <= 0.90, "POSIBLE CARA en vol (GARCH <
implícita)",
                                              "SIN VENTAJA clara (|diff| <
10%)")))

tabla_senal <- data.frame(
  Accion = tickers,
  Sigma_GARCH_34d = as.numeric(vol_garch_prom_34d[tickers]),
  Sigma_implicita_TV = as.numeric(vol_impl_tv[tickers]),
  Razon = as.numeric(razon_vol),
  Senal_10pct = as.character(senal_vol),
  row.names = NULL
)

print(tabla_senal)

# 2) Costo y puntos de equilibrio del straddle (call + put al mismo strike)
# En esta tarea, el strike relevante por acción es: 200 (NVDA), 500 (AMD),
185 (XOM)
K_map <- c(NVDA = 200, AMD = 500, XOM = 185)

# Necesitas bid/ask para call y put por acción. Aquí los guardamos en
listas.
# Si ya los tienes en variables separadas, reemplaza estos NA por tus
valores.
bid_call <- c(NVDA = NA_real_, AMD = NA_real_, XOM = NA_real_)
ask call <- c(NVDA = NA real , AMD = NA real , XOM = NA real )
bid put  <- c(NVDA = NA real , AMD = NA real , XOM = NA real )
ask_put  <- c(NVDA = NA_real_, AMD = NA_real_, XOM = NA_real_)

# Costo del straddle usando el ask (costo de entrada)
costo straddle <- ask call + ask put

eq_inf <- S0 - costo_straddle
eq_sup <- S0 + costo_straddle

tabla_straddle <- data.frame(
```

```r
  Accion = tickers,
  K = as.numeric(K_map[tickers]),
  Spot hoy = as.numeric(S0[tickers]),
  Costo_straddle_ask = as.numeric(costo_straddle[tickers]),
  Equilibrio_inferior = as.numeric(eq_inf[tickers]),
  Equilibrio_superior = as.numeric(eq_sup[tickers]),
  row.names = NULL
)

print(tabla_straddle)

# 3) Simular 10 días hábiles con GARCH para obtener S_10 y revaluar con el
árbol
# Simulación bajo medida real usando media histórica diaria (como en el
Bloque 4)
simula_S10_garch <- function(S0, x_ret_pct, fit, N, T_var = 10) {
  x_ret_pct <- as.numeric(x_ret_pct)
  x ret pct <- x ret pct[is.finite(x ret pct)]

  mu <- mean(x_ret_pct)          # drift histórico en % diario
  omega <- fit$omega
  alpha <- fit$alpha
  beta  <- fit$beta

  S <- rep(S0, N)
  h <- rep(garch_htoday(x_ret_pct, fit), N)

  for (t in 1:T_var) {
    z <- rnorm(N)
    e <- sqrt(h) * z
    rday <- mu + e
    S <- S * exp(rday / 100)
    h <- omega + alpha * (e^2) + beta * h
  }
  S
}

# Revaluación a 10 días: usar CRR americano con sigma implícita
(TradingView) y tiempo remanente
Nsteps_reval <- 500

revalua_opcion_arbol <- function(S_now, K, tipo, sigma_ann, r, q, T_rem,
Nsteps) {
  # Usa el mismo pricer del Bloque 5
  crr price(S0 = S now, K = K, r = r, q = q, sigma ann = sigma ann,
            T = T rem, Nsteps = Nsteps, type = tipo, american = TRUE)
}

# Posiciones a 2 semanas: las de la tarea (NVDA call 200 x2, AMD put 500
x1, XOM call 185 x3)
pos <- data.frame(
  Accion = c("NVDA", "AMD", "XOM"),
  Tipo = c("call", "put", "call"),
  K = c(200, 500, 185),
  Contratos = c(2, 1, 3),
  stringsAsFactors = FALSE
)
mult <- 100

# Precio pagado al inicio: usar el ask de mercado de cada opción
```

```
# Debes llenar estos asks (USD por acción).
ask_pos <- c(NVDA = NA_real_, AMD = NA_real_, XOM = NA_real_)
names(ask pos) <- pos$Accion

# Correr simulación y revaluación por posición
res_2w <- lapply(1:nrow(pos), function(i) {
  tk <- pos$Accion[i]
  tipo <- pos$Tipo[i]
  K <- pos$K[i]
  contratos <- pos$Contratos[i]

  # Simular precio a 10 días
  S10 <- simula S10 garch(S0 = S0[tk], x ret pct = ret[[tk]], fit =
ajustes[[tk]],
                          N = Npaths, T_var = T_2w_days)

  # Revaluar cada trayectoria con árbol (tiempo remanente)
  sigma use <- vol impl tv[tk]
  q_use <- q_yield[tk]

  V10 <- vapply(S10, function(s) {
    revalua_opcion_arbol(S_now = s, K = K, tipo = tipo, sigma_ann =
sigma_use,
                         r = r, q = q_use, T_rem = T_rem, Nsteps =
Nsteps_reval)
  }, numeric(1))

  # P&L por contrato y por posición
  precio_pagado <- ask_pos[tk]
  pnl_por_accion <- V10 - precio_pagado
  pnl_pos <- pnl_por_accion * contratos * mult

  data.frame(
    Accion = tk,
    Tipo = tipo,
    K = K,
    Contratos = contratos,
    Costo_total_USD = precio_pagado * contratos * mult,
    PnL_esperado_2sem_USD = mean(pnl_pos, na.rm = TRUE),
    Rentabilidad_esperada = mean(pnl_pos, na.rm = TRUE) / (precio_pagado *
contratos * mult),
    Prob_de_ganar = mean(pnl_pos > 0, na.rm = TRUE),
    P5_USD = as.numeric(quantile(pnl_pos, probs = 0.05, na.rm = TRUE)),
    P95_USD = as.numeric(quantile(pnl_pos, probs = 0.95, na.rm = TRUE)),
    row.names = NULL
  )
})

tabla_2w <- do.call(rbind, res_2w)
print(tabla_2w)
```

Notas rápidas para que te corra “tal cual”:

1. Llena `vol_impl_tv` (NVDA, AMD, XOM) en % anual.
2. Llena `ask_call`, `ask_put` para el straddle, y `ask_pos` para cada posición (asks de NVDA call 200, AMD put 500, XOM call 185) en USD por acción.

3. Si quieres que la revaluación a 10 días use otra sigma (GARCH o histórica) en vez de implícita, cambia `sigma_use <- vol_impl_tv[tk]`.

Thinking
[DONE]

2/11/2026 - 4:40:29 p.m.

Junta todos los bloques en un solo Rmd ordenado con títulos

ChatGPT (GPT 5.2) •2/11/2026 - 4:40:29 p.m.

````
---
title: "Tarea Clase 6: GARCH, Monte Carlo y Árboles Binomiales (NVDA, AMD, XOM)"
author: "Equipo"
date: "`r format(Sys.Date())`"
output: html_document
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE, warning = FALSE, message = FALSE)
````

1. Lectura de datos y rendimientos

```
## Bloque 1: Lectura de datos, rendimientos logarítmicos y volatilidad (R base)

# 1) Leer CSV (ajusta el nombre del archivo si es necesario)
datos <- read.csv("precios clase6.csv", stringsAsFactors = FALSE)

# 2) Convertir y ordenar por fecha
datos$Date <- as.Date(datos$Date)   # si tu fecha viene como "YYYY-MM-DD"
datos <- datos[order(datos$Date), ]

# 3) Extraer precios de las tres acciones (ajusta nombres de columnas si difieren)
precios <- datos[, c("NVDA", "AMD", "XOM")]
precios <- as.data.frame(lapply(precios, as.numeric))

# 4) Calcular rendimientos logarítmicos diarios en porcentaje
# r_t = 100 * ln(P_t / P_{t-1})
ret <- as.data.frame(lapply(precios, function(p) 100 * diff(log(p))))
ret$Date <- datos$Date[-1]

# 5) Volatilidad anualizada (252 días hábiles)
vol_anual <- function(x) sd(x, na.rm = TRUE) * sqrt(252)

# 6) Vol anualizada: últimos 30 días hábiles y último año hábil (252)
```
```

```
n <- nrow(ret)
ult30 <- max(1, n - 30 + 1):n
ulty  <- max(1, n - 252 + 1):n

vol30 <- sapply(ret[, c("NVDA","AMD","XOM")], function(x)
vol_anual(x[ult30]))
vol1y <- sapply(ret[, c("NVDA","AMD","XOM")], function(x)
vol anual(x[ulty]))

tabla_vol <- data.frame(
  Accion = c("NVDA", "AMD", "XOM"),
  Vol_30d_anual = as.numeric(vol30),
  Vol 1y anual  = as.numeric(vol1y),
  row.names = NULL
)

print(tabla_vol)
```

2. Estimación GARCH(1,1) por MLE (innovaciones normales)

```
## Bloque 2: Estimación GARCH(1,1) por MLE con innovaciones normales (R
base)
## Rendimientos en porcentaje diario

fit garch11 norm <- function(x) {
  x <- as.numeric(x)
  x <- x[is.finite(x)]
  mu <- mean(x)
  e  <- x - mu
  n  <- length(e)

  # Varianza inicial
  h0 <- var(e)
  if (!is.finite(h0) || h0 <= 0) h0 <- 1

  # Negativa de la log-verosimilitud con penalización para alpha+beta<1
  nll <- function(par) {
    omega <- par[1]
    alpha <- par[2]
    beta  <- par[3]

    if (omega <= 0 || alpha < 0 || beta < 0) return(1e12)
    if (alpha + beta >= 0.999999) return(1e12 + 1e8 * (alpha + beta -
0.999999)^2)

    h <- numeric(n)
    h[1] <- h0

    for (t in 2:n) {
      h[t] <- omega + alpha * e[t - 1]^2 + beta * h[t - 1]
      if (!is.finite(h[t]) || h[t] <= 0) return(1e12)
    }

    ll <- -0.5 * (log(2 * pi) + log(h) + (e^2) / h)
    -sum(ll)
```

```r
  }

  # Inicialización
  alpha0 <- 0.05
  beta0  <- 0.90
  omega0 <- max(1e-6, h0 * (1 - alpha0 - beta0))

  opt <- optim(
    par    = c(omega0, alpha0, beta0),
    fn     = nll,
    method = "L-BFGS-B",
    lower  = c(1e-12, 0, 0),
    upper  = c(Inf, 1, 1)
  )

  par <- opt$par
  omega <- par[1]; alpha <- par[2]; beta <- par[3]
  pers  <- alpha + beta

  # Reconstruir h_t para "volatilidad de hoy"
  h <- numeric(n)
  h[1] <- h0
  for (t in 2:n) h[t] <- omega + alpha * e[t - 1]^2 + beta * h[t - 1]

  h_lr <- omega / (1 - pers)
  vol_lr_ann    <- sqrt(h_lr) * sqrt(252)
  vol_today_ann <- sqrt(h[n]) * sqrt(252)

  half_life <- if (pers > 0 && pers < 1) log(0.5) / log(pers) else NA_real_

  list(
    omega = omega, alpha = alpha, beta = beta,
    persistencia = pers,
    vol_lr_ann = vol_lr_ann,
    vol_today_ann = vol_today_ann,
    half_life = half_life,
    converged = (opt$convergence == 0)
  )
}

tickers <- c("NVDA", "AMD", "XOM")
ajustes <- lapply(tickers, function(tk) fit_garch11_norm(ret[[tk]]))
names(ajustes) <- tickers

tabla_garch <- data.frame(
  Accion = tickers,
  omega = sapply(ajustes, `[[`, "omega"),
  alpha = sapply(ajustes, `[[`, "alpha"),
  beta  = sapply(ajustes, `[[`, "beta"),
  Persistencia = sapply(ajustes, `[[`, "persistencia"),
  Vol_largo_plazo_anual = sapply(ajustes, `[[`, "vol_lr_ann"),
  Vol_hoy_anual = sapply(ajustes, `[[`, "vol_today_ann"),
  Vida_media_shock_dias = sapply(ajustes, `[[`, "half_life"),
  Convergio = sapply(ajustes, `[[`, "converged"),
  row.names = NULL
)

print(tabla_garch)
```

3. EWMA y pronóstico de varianza GARCH por reversión a la media

```
## Bloque 3: EWMA (lambda 0.94) y pronóstico GARCH por reversión a la media

tickers <- c("NVDA", "AMD", "XOM")
lambda <- 0.94

ewma_vol_hoy <- function(x, lambda = 0.94) {
  x <- as.numeric(x)
  x <- x[is.finite(x)]
  n <- length(x)
  if (n < 2) return(NA_real_)

  e <- x - mean(x)
  s2 <- var(e)
  if (!is.finite(s2) || s2 <= 0) s2 <- 1

  for (t in 2:n) s2 <- lambda * s2 + (1 - lambda) * e[t - 1]^2
  sqrt(s2) * sqrt(252)
}

vol_ewma_hoy <- sapply(tickers, function(tk) ewma_vol_hoy(ret[[tk]],
lambda))

garch_forecast_var <- function(x, fit, horizons = c(10, 30, 34, 126, 252))
{
  x <- as.numeric(x)
  x <- x[is.finite(x)]
  mu <- mean(x)
  e  <- x - mu
  n  <- length(e)

  omega <- fit$omega
  alpha <- fit$alpha
  beta  <- fit$beta
  pers  <- alpha + beta

  h0 <- var(e)
  if (!is.finite(h0) || h0 <= 0) h0 <- 1

  h <- numeric(n)
  h[1] <- h0
  for (t in 2:n) h[t] <- omega + alpha * e[t - 1]^2 + beta * h[t - 1]
  h_today <- h[n]

  h_bar <- omega / (1 - pers)

  fvar <- sapply(horizons, function(H) {
    h_bar + (pers^H) * (h_today - h_bar)
  })

  list(h_today = h_today, h_bar = h_bar, pers = pers, fvar = fvar)
}

horizons <- c(10, 30, 34, 126, 252)
```

```r
vol_garch_h <- matrix(NA_real_, nrow = length(tickers), ncol =
length(horizons),
                      dimnames = list(tickers, paste0(horizons, "d")))
vol_garch_prom_34d <- setNames(rep(NA_real_, length(tickers)), tickers)

for (tk in tickers) {
  fc <- garch_forecast_var(ret[[tk]], ajustes[[tk]], horizons)

  vol_garch_h[tk, ] <- sqrt(fc$fvar) * sqrt(252)

  # Volatilidad promedio esperada hasta vencimiento (T = 34 días, como en
la tarea)
  Tpath <- 34
  fvar_path <- fc$h_bar + (fc$pers^(1:Tpath)) * (fc$h_today - fc$h_bar)
  vol_garch_prom_34d[tk] <- sqrt(mean(fvar_path)) * sqrt(252)
}

# Volatilidad implícita TradingView en % anual (llenar)
vol_impl_tv <- c(NVDA = NA_real_, AMD = NA_real_, XOM = NA_real_)

razon <- vol_garch_prom_34d / vol_impl_tv
senal <- ifelse(is.na(razon), NA_character_,
                ifelse(razon >= 1.10, "GARCH > implícita (posible barata en
vol)",
                ifelse(razon <= 0.90, "GARCH < implícita (posible cara en
vol)",
                                      "Sin ventaja clara (|diff| < 10%)")))

tabla_ewma <- data.frame(
  Accion = tickers,
  Vol_EWMA_hoy_anual = as.numeric(vol_ewma_hoy),
  row.names = NULL
)

tabla_garch_plazos <- data.frame(
  Accion = tickers,
  Vol_GARCH_10d  = as.numeric(vol_garch_h[, "10d"]),
  Vol_GARCH_30d  = as.numeric(vol_garch_h[, "30d"]),
  Vol_GARCH_34d  = as.numeric(vol_garch_h[, "34d"]),
  Vol_GARCH_126d = as.numeric(vol_garch_h[, "126d"]),
  Vol_GARCH_252d = as.numeric(vol_garch_h[, "252d"]),
  Vol_prom_esperada_34d = as.numeric(vol_garch_prom_34d),
  Sigma_implicita_TV = as.numeric(vol_impl_tv),
  Razon_GARCH_sobre_Impl = as.numeric(razon),
  Senal_10pct = as.character(senal),
  row.names = NULL
)

print(tabla_ewma)
print(tabla_garch_plazos)
```

4. Monte Carlo con GARCH(1,1): distribución a vencimiento y VaR 10 días

```r
## Bloque 4: Monte Carlo con GARCH(1,1)

set.seed(123)
N <- 100000
T_venc <- 34
T_var <- 10
tickers <- c("NVDA", "AMD", "XOM")

S0 <- as.numeric(datos[nrow(datos), tickers])
names(S0) <- tickers

garch_htoday <- function(x, fit) {
  x <- as.numeric(x)
  x <- x[is.finite(x)]
  mu <- mean(x)
  e  <- x - mu
  n  <- length(e)

  omega <- fit$omega
  alpha <- fit$alpha
  beta  <- fit$beta

  h0 <- var(e)
  if (!is.finite(h0) || h0 <= 0) h0 <- 1

  h <- numeric(n)
  h[1] <- h0
  for (t in 2:n) h[t] <- omega + alpha * e[t - 1]^2 + beta * h[t - 1]
  h[n]
}

simula_precio_garch <- function(S0, x_ret_pct, fit, N, T_venc, T_var) {
  x_ret_pct <- as.numeric(x_ret_pct)
  x_ret_pct <- x_ret_pct[is.finite(x_ret_pct)]

  mu <- mean(x ret pct)

  omega <- fit$omega
  alpha <- fit$alpha
  beta  <- fit$beta

  S <- rep(S0, N)
  h <- rep(garch_htoday(x_ret_pct, fit), N)

  S10 <- rep(NA real , N)

  for (t in 1:T_venc) {
    z <- rnorm(N)
    e <- sqrt(h) * z
    r <- mu + e
    S <- S * exp(r / 100)
    h <- omega + alpha * (e^2) + beta * h
    if (t == T_var) S10 <- S
  }

  p <- as.numeric(quantile(S, probs = c(0.05, 0.25, 0.50, 0.75, 0.95),
na.rm = TRUE))
  names(p) <- c("P5","P25","P50","P75","P95")

  prob_subir <- mean(S > S0)
```

```
  loss10 <- (S0 - S10) / S0
  var99_10d <- as.numeric(quantile(loss10, probs = 0.99, na.rm = TRUE))

  list(P5 = p["P5"], P25 = p["P25"], P50 = p["P50"], P75 = p["P75"], P95 =
p["P95"],
       prob_subir = prob_subir, var99_10d = var99_10d)
}

res <- lapply(tickers, function(tk) {
  simula_precio_garch(S0 = S0[tk], x_ret_pct = ret[[tk]], fit =
ajustes[[tk]],
                      N = N, T_venc = T_venc, T_var = T_var)
})
names(res) <- tickers

tabla_mc <- data.frame(
  Accion = tickers,
  Precio_hoy = as.numeric(S0[tickers]),
  P5  = sapply(res, `[[`, "P5"),
  P25 = sapply(res, `[[`, "P25"),
  P50 = sapply(res, `[[`, "P50"),
  P75 = sapply(res, `[[`, "P75"),
  P95 = sapply(res, `[[`, "P95"),
  Prob_de_subir = sapply(res, `[[`, "prob_subir"),
  VaR_99pct_10d = sapply(res, `[[`, "var99_10d"),
  row.names = NULL
)

print(tabla_mc)
```

5. Árbol binomial CRR americano (500 pasos) con dividend yield continuo q

```
## Bloque 5: Árbol binomial CRR (americano y europeo)

r <- 0.0394
T <- 34/365
Nsteps <- 500

q_yield <- c(NVDA = 0.0047, AMD = 0.0000, XOM = 0.0225)

inst <- data.frame(
  Accion = c("NVDA", "AMD", "XOM"),
  Tipo   = c("call", "put", "call"),
  K      = c(200, 500, 185),
  stringsAsFactors = FALSE
)

S0 <- as.numeric(datos[nrow(datos), inst$Accion])
names(S0) <- inst$Accion

vol_hist_1y <- sapply(inst$Accion, function(tk) sd(ret[[tk]], na.rm = TRUE)
* sqrt(252))  # en %
vol_garch_34 <- vol_garch_prom_34d[inst$Accion]
# en %
```

```
# Entradas de TradingView y mercado (llenar)
vol impl tv <- c(NVDA = NA real , AMD = NA real , XOM = NA real )  # %
anual

bid_mkt <- c(NVDA = NA_real_, AMD = NA_real_, XOM = NA_real_)      # USD
por acción
ask mkt <- c(NVDA = NA real , AMD = NA real , XOM = NA real )      # USD
por acción

crr_price <- function(S0, K, r, q, sigma_ann, T, Nsteps, type =
c("call","put"), american = TRUE) {
  type <- match.arg(type)

  sigma <- sigma_ann / 100
  dt <- T / Nsteps
  u <- exp(sigma * sqrt(dt))
  d <- 1 / u
  disc <- exp(-r * dt)
  p <- (exp((r - q) * dt) - d) / (u - d)

  if (!is.finite(p) || p < 0 || p > 1) return(NA_real_)

  j <- 0:Nsteps
  ST <- S0 * (u^j) * (d^(Nsteps - j))

  if (type == "call") V <- pmax(ST - K, 0) else V <- pmax(K - ST, 0)

  for (i in (Nsteps - 1):0) {
    V <- disc * (p * V[2:(i + 2)] + (1 - p) * V[1:(i + 1)])

    if (american) {
      j <- 0:i
      Sij <- S0 * (u^j) * (d^(i - j))
      if (type == "call") intrinsic <- pmax(Sij - K, 0) else intrinsic <-
pmax(K - Sij, 0)
      V <- pmax(V, intrinsic)
    }
  }
  V[1]
}

valua_instrumento <- function(tk, tipo, K, r, q, T, Nsteps,
                              sigma_hist, sigma_garch, sigma_impl, bid,
ask) {

  sigmas <- c(Historica_1y = sigma_hist, GARCH_34d = sigma_garch,
Implicita_TV = sigma_impl)

  out <- lapply(names(sigmas), function(nm) {
    sig <- sigmas[[nm]]

    price_eur <- crr_price(S0[tk], K, r, q, sig, T, Nsteps, type = tipo,
american = FALSE)
    price ame <- crr price(S0[tk], K, r, q, sig, T, Nsteps, type = tipo,
american = TRUE)

    data.frame(
      Accion = tk,
      Tipo = tipo,
```

```
      K = K,
      Sigma_fuente = nm,
      Sigma_anual_pct = sig,
      Precio_Europeo = price_eur,
      Precio_Americano = price_ame,
      Prima_ejercicio_anticipado = price_ame - price_eur,
      Bid = bid,
      Ask = ask,
      row.names = NULL,
      stringsAsFactors = FALSE
    )
  })

  do.call(rbind, out)
}

tabla_arbol <- do.call(rbind, lapply(1:nrow(inst), function(i) {
  tk <- inst$Accion[i]
  valua_instrumento(
    tk = tk,
    tipo = inst$Tipo[i],
    K = inst$K[i],
    r = r,
    q = q_yield[tk],
    T = T,
    Nsteps = Nsteps,
    sigma_hist = vol_hist_1y[tk],
    sigma_garch = vol_garch_34[tk],
    sigma_impl = vol_impl_tv[tk],
    bid = bid_mkt[tk],
    ask = ask_mkt[tk]
  )
}))

print(tabla_arbol)
```

6. Griegas desde el árbol y nodos con decisión (ejercer o esperar)

```
## Bloque 6: Delta, gamma, theta desde nodos; vega y rho por diferencias
finitas
## Además: nodos de pasos 1, 2 y 4 con decisión ejercer o esperar

pos <- data.frame(
  Accion = c("NVDA", "AMD", "XOM"),
  Tipo   = c("call", "put", "call"),
  K      = c(200, 500, 185),
  Contratos = c(2, 1, 3),
  stringsAsFactors = FALSE
)
mult <- 100

bump_sigma_pto <- 1.0
bump_r_pto     <- 0.01
```

```r
crr_american_with_nodes <- function(S0, K, r, q, sigma_ann, T, Nsteps, type
= c("call","put"),
                                    keep steps = c(1,2,4)) {
  type <- match.arg(type)

  sigma <- sigma_ann / 100
  dt <- T / Nsteps
  u <- exp(sigma * sqrt(dt))
  d <- 1 / u
  disc <- exp(-r * dt)
  p <- (exp((r - q) * dt) - d) / (u - d)

  if (!is.finite(p) || p < 0 || p > 1) return(list(price = NA real , nodes
= NULL, dt = dt))

  j <- 0:Nsteps
  ST <- S0 * (u^j) * (d^(Nsteps - j))
  if (type == "call") V <- pmax(ST - K, 0) else V <- pmax(K - ST, 0)

  nodes <- list()

  for (i in (Nsteps - 1):0) {
    V_cont <- disc * (p * V[2:(i + 2)] + (1 - p) * V[1:(i + 1)])

    j <- 0:i
    Sij <- S0 * (u^j) * (d^(i - j))
    if (type == "call") intrinsic <- pmax(Sij - K, 0) else intrinsic <-
pmax(K - Sij, 0)

    exercise <- intrinsic > V_cont
    V <- pmax(V_cont, intrinsic)

    if (i %in% keep_steps) {
      nodes[[paste0("step_", i)]] <- data.frame(
        step = i,
        j = j,
        S = Sij,
        Continuar = V_cont,
        Intrinseco = intrinsic,
        Valor_americano = V,
        Decision = ifelse(exercise, "EJERCER", "ESPERAR"),
        row.names = NULL
      )
    }
  }

  list(price = V[1], nodes = nodes, dt = dt, u = u, d = d, p = p)
}

crr greeks <- function(S0, K, r, q, sigma ann, T, Nsteps, type =
c("call","put")) {
  type <- match.arg(type)

  out <- crr_american_with_nodes(S0, K, r, q, sigma_ann, T, Nsteps, type,
keep steps = c(1,2))
  if (!is.finite(out$price)) {
    return(list(delta = NA, gamma = NA, theta = NA, vega = NA, rho = NA))
  }

  dt <- out$dt
```

```
  n1 <- out$nodes[["step_1"]]
  n2 <- out$nodes[["step_2"]]

  Vd <- n1$Valor_americano[n1$j == 0]
  Vu <- n1$Valor_americano[n1$j == 1]
  Sd <- n1$S[n1$j == 0]
  Su <- n1$S[n1$j == 1]
  delta <- (Vu - Vd) / (Su - Sd)

  Vdd <- n2$Valor_americano[n2$j == 0]
  Vud <- n2$Valor_americano[n2$j == 1]
  Vuu <- n2$Valor_americano[n2$j == 2]

  Sdd <- n2$S[n2$j == 0]
  Sud <- n2$S[n2$j == 1]
  Suu <- n2$S[n2$j == 2]

  delta_up   <- (Vuu - Vud) / (Suu - Sud)
  delta_down <- (Vud - Vdd) / (Sud - Sdd)
  gamma <- (delta_up - delta_down) / ((Suu - Sdd) / 2)

  V0 <- out$price
  theta_per_year <- (Vud - V0) / (2 * dt)
  theta_per_day <- theta_per_year / 365

  price_up <- crr_american_with_nodes(S0, K, r, q, sigma_ann +
bump_sigma_pto, T, Nsteps, type,
                                      keep_steps = integer(0))$price
  price_dn <- crr_american_with_nodes(S0, K, r, q, sigma_ann -
bump_sigma_pto, T, Nsteps, type,
                                      keep_steps = integer(0))$price
  vega <- (price_up - price_dn) / (2 * bump_sigma_pto)

  price_ru <- crr_american_with_nodes(S0, K, r + bump_r_pto, q, sigma_ann,
T, Nsteps, type,
                                      keep steps = integer(0))$price
  price rd <- crr american with nodes(S0, K, r - bump r pto, q, sigma ann,
T, Nsteps, type,
                                      keep_steps = integer(0))$price
  rho <- (price_ru - price_rd) / (2 * bump_r_pto)

  list(delta = delta, gamma = gamma, theta = theta_per_day, vega = vega,
rho = rho)
}

# Griegas usando sigma implícita (TradingView) como base
tabla_greeks <- do.call(rbind, lapply(1:nrow(pos), function(i) {
  tk <- pos$Accion[i]
  tipo <- pos$Tipo[i]
  K <- pos$K[i]
  contratos <- pos$Contratos[i]

  sigma_use <- vol_impl_tv[tk]

  g <- crr greeks(S0 = S0[tk], K = K, r = r, q = q yield[tk],
                  sigma ann = sigma use, T = T, Nsteps = Nsteps, type =
tipo)

  data.frame(
    Accion = tk,
```

```
    Tipo = tipo,
    K = K,
    Sigma usada pct = sigma use,
    Contratos = contratos,
    Delta_opcion = g$delta,
    Gamma_opcion = g$gamma,
    Theta_opcion_USD_dia = g$theta,
    Vega opcion USD por 1pto = g$vega,
    Rho_opcion_USD_por_1pto = g$rho,
    Delta_posicion_acciones = g$delta * contratos * mult,
    Gamma_posicion = g$gamma * contratos * mult,
    Theta_posicion_USD_dia = g$theta * contratos * mult,
    Vega posicion USD por 1pto = g$vega * contratos * mult,
    Rho_posicion_USD_por_1pto = g$rho * contratos * mult,
    row.names = NULL
  )
}))

print(tabla_greeks)

# Nodos de pasos 1, 2 y 4 con decisión ejercer o esperar
for (i in 1:nrow(pos)) {
  tk <- pos$Accion[i]
  tipo <- pos$Tipo[i]
  K <- pos$K[i]
  sigma_use <- vol_impl_tv[tk]

  cat("\n========================\n")
  cat("Nodos y decisión:", tk, tipo, "K =", K, "sigma =", sigma_use, "%\n")
  cat("========================\n")

  dbg <- crr_american_with_nodes(S0 = S0[tk], K = K, r = r, q =
q_yield[tk],
                                 sigma_ann = sigma_use, T = T, Nsteps =
Nsteps,
                                 type = tipo, keep steps = c(1,2,4))

  print(dbg$nodes[["step_1"]])
  print(dbg$nodes[["step_2"]])
  print(dbg$nodes[["step_4"]])
}
```

7. Señal de volatilidad, straddle y rentabilidad a 2 semanas (revaluación con árbol)

```
## Bloque 7: Señal de volatilidad, costo y puntos de equilibrio del
straddle,
## y rentabilidad a 2 semanas (10 días hábiles) simulando precio con GARCH
y revaluando con CRR

tickers <- c("NVDA", "AMD", "XOM")

T_total_days <- 34
```

```r
T_total <- T_total_days / 365

T 2w days <- 10
T_rem_days <- T_total_days - T_2w_days
T_rem <- T_rem_days / 365

Npaths <- 100000
set.seed(123)

S0 <- as.numeric(datos[nrow(datos), tickers])
names(S0) <- tickers

# 1) Señal de volatilidad: razón sigma GARCH / sigma implícita
razon_vol <- vol_garch_prom_34d[tickers] / vol_impl_tv[tickers]
senal_vol <- ifelse(is.na(razon_vol), NA_character_,
                    ifelse(razon_vol >= 1.10, "POSIBLE BARATA en vol (GARCH
> implícita)",
                    ifelse(razon vol <= 0.90, "POSIBLE CARA en vol (GARCH <
implícita)",
                                              "SIN VENTAJA clara (|diff| <
10%)")))

tabla_senal <- data.frame(
  Accion = tickers,
  Sigma_GARCH_34d = as.numeric(vol_garch_prom_34d[tickers]),
  Sigma_implicita_TV = as.numeric(vol_impl_tv[tickers]),
  Razon = as.numeric(razon_vol),
  Senal_10pct = as.character(senal_vol),
  row.names = NULL
)
print(tabla_senal)

# 2) Straddle: costo y puntos de equilibrio
K_map <- c(NVDA = 200, AMD = 500, XOM = 185)

# Bid y ask para CALL y PUT del straddle (llenar, USD por acción)
bid call <- c(NVDA = NA real , AMD = NA real , XOM = NA real )
ask_call <- c(NVDA = NA_real_, AMD = NA_real_, XOM = NA_real_)
bid_put  <- c(NVDA = NA_real_, AMD = NA_real_, XOM = NA_real_)
ask_put  <- c(NVDA = NA_real_, AMD = NA_real_, XOM = NA_real_)

costo_straddle <- ask_call + ask_put
eq_inf <- S0 - costo_straddle
eq_sup <- S0 + costo_straddle

tabla straddle <- data.frame(
  Accion = tickers,
  K = as.numeric(K_map[tickers]),
  Spot_hoy = as.numeric(S0[tickers]),
  Costo straddle ask = as.numeric(costo straddle[tickers]),
  Equilibrio inferior = as.numeric(eq inf[tickers]),
  Equilibrio_superior = as.numeric(eq_sup[tickers]),
  row.names = NULL
)
print(tabla straddle)

# 3) Simular 10 días con GARCH para obtener S_10
simula_S10_garch <- function(S0, x_ret_pct, fit, N, T_var = 10) {
  x_ret_pct <- as.numeric(x_ret_pct)
  x_ret_pct <- x_ret_pct[is.finite(x_ret_pct)]
```

```
  mu <- mean(x_ret_pct)

  omega <- fit$omega
  alpha <- fit$alpha
  beta  <- fit$beta

  S <- rep(S0, N)
  h <- rep(garch_htoday(x_ret_pct, fit), N)

  for (t in 1:T_var) {
    z <- rnorm(N)
    e <- sqrt(h) * z
    rday <- mu + e
    S <- S * exp(rday / 100)
    h <- omega + alpha * (e^2) + beta * h
  }
  S
}

# 4) Revaluación a 10 días: CRR americano con tiempo remanente y sigma
implícita
Nsteps_reval <- 500

revalua_opcion_arbol <- function(S_now, K, tipo, sigma_ann, r, q, T_rem,
Nsteps) {
  crr_price(S0 = S_now, K = K, r = r, q = q, sigma_ann = sigma_ann,
            T = T_rem, Nsteps = Nsteps, type = tipo, american = TRUE)
}

# Posiciones del equipo
pos <- data.frame(
  Accion = c("NVDA", "AMD", "XOM"),
  Tipo = c("call", "put", "call"),
  K = c(200, 500, 185),
  Contratos = c(2, 1, 3),
  stringsAsFactors = FALSE
)
mult <- 100

# Ask de cada posición (llenar, USD por acción)
ask_pos <- c(NVDA = NA_real_, AMD = NA_real_, XOM = NA_real_)

res_2w <- lapply(1:nrow(pos), function(i) {
  tk <- pos$Accion[i]
  tipo <- pos$Tipo[i]
  K <- pos$K[i]
  contratos <- pos$Contratos[i]

  S10 <- simula S10 garch(S0 = S0[tk], x ret pct = ret[[tk]], fit =
ajustes[[tk]],
                          N = Npaths, T_var = T_2w_days)

  sigma_use <- vol_impl_tv[tk]
  q use <- q yield[tk]

  V10 <- vapply(S10, function(s) {
    revalua_opcion_arbol(S_now = s, K = K, tipo = tipo, sigma_ann =
sigma_use,
```

```
                          r = r, q = q_use, T_rem = T_rem, Nsteps =
Nsteps_reval)
  }, numeric(1))

  precio_pagado <- ask_pos[tk]
  pnl_por_accion <- V10 - precio_pagado
  pnl_pos <- pnl_por_accion * contratos * mult

  data.frame(
    Accion = tk,
    Tipo = tipo,
    K = K,
    Contratos = contratos,
    Costo_total_USD = precio_pagado * contratos * mult,
    PnL_esperado_2sem_USD = mean(pnl_pos, na.rm = TRUE),
    Rentabilidad_esperada = mean(pnl_pos, na.rm = TRUE) / (precio_pagado *
contratos * mult),
    Prob de ganar = mean(pnl pos > 0, na.rm = TRUE),
    P5_USD = as.numeric(quantile(pnl_pos, probs = 0.05, na.rm = TRUE)),
    P95_USD = as.numeric(quantile(pnl_pos, probs = 0.95, na.rm = TRUE)),
    row.names = NULL
  )
})

tabla_2w <- do.call(rbind, res_2w)
print(tabla_2w)
```

Thinking

[DONE]

2/11/2026 - 5:13:16 p.m.

Revisé tu código con los datos reales y encontré errores. Corrígelos y devuélveme los bloques 3 a 7 completos: (1) El tiempo al vencimiento de las opciones debe ser T = 49/365 (días naturales del 30-oct al 18-dic); los 34 son días hábiles y solo se usan para simular trayectorias. En el Bloque 7 el tiempo remanente a 10 días hábiles es (49 menos 14)/365. (2) La volatilidad histórica de 1 año debe usar solo los últimos 252 rendimientos, no toda la muestra. (3) El Monte Carlo no debe sumar el rendimiento medio histórico como deriva; usa deriva neutral con la corrección de Itô (-0.5 sigma al cuadrado). (4) El rho se debe expresar por +1 punto porcentual, así que divide entre 100. (5) La varianza GARCH de partida debe ser la del día siguiente, omega + alpha por el último rendimiento al cuadrado + beta por la última varianza. Explica cada corrección. No uses guiones largos.



2/11/2026 - 7:53:29 p.m.

Revisé tu código con los datos reales y encontré errores. Corrígelos y devuélveme los bloques 3 a 7 completos: (1) El tiempo al vencimiento de las opciones debe ser T = 49/365 (días naturales del 30-oct al 18-dic); los 34 son días hábiles y solo se usan para simular

trayectorias. En el Bloque 7 el tiempo remanente a 10 días hábiles es (49 menos 14)/365. (2) La volatilidad histórica de 1 año debe usar solo los últimos 252 rendimientos, no toda la muestra. (3) El Monte Carlo no debe sumar el rendimiento medio histórico como deriva; usa deriva neutral con la corrección de Itô (-0.5 sigma al cuadrado). (4) El rho se debe expresar por +1 punto porcentual, así que divide entre 100. (5) La varianza GARCH de partida debe ser la del día siguiente, omega + alpha por el último rendimiento al cuadrado + beta por la última varianza. Explica cada corrección. No uses guiones largos.

ChatGPT (GPT 5.2) •2/11/2026 - 7:53:30 p.m.

A continuación van las correcciones explicadas y luego los **Bloques 3 a 7 completos** ya ajustados.

Explicación de cada corrección (lo que cambió y por qué)

1. **Tiempo al vencimiento**
   - Para valuar opciones en el árbol: **T = 49/365** (días naturales del 30-oct al 18-dic).
   - Los **34** se usan solo como aproximación de **días hábiles** para simular trayectorias hasta vencimiento.
   - En el Bloque 7, a "2 semanas" son **10 días hábiles ≈ 14 naturales**, así que el tiempo remanente es **(49 − 14)/365**.
2. **Volatilidad histórica 1 año**
   - Se debe calcular con **solo los últimos 252 rendimientos** (ventana móvil de 1 año hábil), no con toda la serie.
3. **Monte Carlo sin deriva histórica**
   - Para consistencia de valuación y comparabilidad, simulamos bajo medida neutral al riesgo:
     $r_{t+1} = (r-q)\Delta t - \tfrac{1}{2}h_t + \sqrt{h_t}z$
     donde h_t es la varianza diaria (en escala decimal) y $\Delta t = 1/252$.
   - Esto elimina el sesgo de "sumar la media histórica" y aplica la corrección de Itô.
4. **Rho por +1 punto porcentual**
   - Si bump en tasa es 0.01 (1 pp), entonces **rho por 1 pp** es:
     $\rho_{1pp} \approx \frac{V(r+0.01)-V(r-0.01)}{2}$
   - Antes se estaba reportando esencialmente "por 1.00 de tasa"; ahora queda por **+1 pp**.
5. **Varianza inicial GARCH: usar la del día siguiente**

- Si al final del historial tienes hth_t y el último residual ete_t, entonces para arrancar pronósticos y simulación debes usar: ht+1=ω+αet2+βht h_{t+1} = \omega + \alpha e_t^2 + \beta h_t
- Esto alinea el "hoy para simular" con la primera varianza que realmente aplica al siguiente paso.

BLOQUE 3 (corregido): EWMA y pronóstico GARCH, y volatilidad promedio hasta vencimiento

```
## Bloque 3: EWMA (lambda 0.94) y pronóstico de varianza GARCH por
reversión a la media
## Correcciones incluidas:
## - Pronósticos arrancan con h_{t+1} (día siguiente)
## - Volatilidad histórica 1 año usa últimos 252 rendimientos
## - 34 se usa como días hábiles para el horizonte de simulación, no como T
en años

tickers <- c("NVDA", "AMD", "XOM")
lambda <- 0.94

# EWMA: volatilidad de hoy (anualizada) en % usando rendimientos en %
diario
ewma_vol_hoy <- function(x, lambda = 0.94) {
  x <- as.numeric(x)
  x <- x[is.finite(x)]
  n <- length(x)
  if (n < 2) return(NA_real_)

  e <- x - mean(x)
  s2 <- var(e)
  if (!is.finite(s2) || s2 <= 0) s2 <- 1

  for (t in 2:n) {
    s2 <- lambda * s2 + (1 - lambda) * e[t - 1]^2
  }

  sqrt(s2) * sqrt(252)
}

# Histórica 1 año: últimos 252 rendimientos
vol_hist_1y <- function(x) {
  x <- as.numeric(x)
  x <- x[is.finite(x)]
  n <- length(x)
  idx <- max(1, n - 252 + 1):n
  sd(x[idx], na.rm = TRUE) * sqrt(252)
}
```

```r
vol_ewma_hoy <- sapply(tickers, function(tk) ewma_vol_hoy(ret[[tk]],
lambda))
vol hist 1y vec <- sapply(tickers, function(tk) vol hist 1y(ret[[tk]]))

# Calcular h_t (última varianza filtrada) y h_{t+1} (día siguiente)
garch_last_h_and_next <- function(x, fit) {
  x <- as.numeric(x)
  x <- x[is.finite(x)]
  mu <- mean(x)
  e  <- x - mu
  n  <- length(e)

  omega <- fit$omega
  alpha <- fit$alpha
  beta  <- fit$beta

  h0 <- var(e)
  if (!is.finite(h0) || h0 <= 0) h0 <- 1

  h <- numeric(n)
  h[1] <- h0
  for (t in 2:n) {
    h[t] <- omega + alpha * e[t - 1]^2 + beta * h[t - 1]
  }

  h_last <- h[n]
  e_last <- e[n]
  h_next <- omega + alpha * e_last^2 + beta * h_last

  list(h_last = h_last, h_next = h_next, mu = mu)
}

# Pronóstico de varianza a horizontes H (días hábiles) usando reversión a
la media
# Arrancando desde h_{t+1}. Para H=1, forecast = h_{t+1}.
garch forecast var from next <- function(x, fit, horizons = c(10, 30, 34,
126, 252)) {
  tmp <- garch_last_h_and_next(x, fit)

  omega <- fit$omega
  alpha <- fit$alpha
  beta  <- fit$beta
  pers  <- alpha + beta

  h bar <- omega / (1 - pers)
  h1 <- tmp$h next

  fvar <- sapply(horizons, function(H) {
    if (H <= 1) return(h1)
    h bar + (pers^(H - 1)) * (h1 - h bar)
  })

  list(h1 = h1, h_bar = h_bar, pers = pers, fvar = fvar)
}

horizons <- c(10, 30, 34, 126, 252)

vol_garch_h <- matrix(NA_real_, nrow = length(tickers), ncol =
length(horizons),
                      dimnames = list(tickers, paste0(horizons, "d")))
```

```r
vol_garch_prom_34d <- setNames(rep(NA_real_, length(tickers)), tickers)

for (tk in tickers) {
  fc <- garch_forecast_var_from_next(ret[[tk]], ajustes[[tk]], horizons)

  # Volatilidad anualizada por horizonte (en %)
  vol_garch_h[tk, ] <- sqrt(fc$fvar) * sqrt(252)

  # Volatilidad promedio esperada hasta vencimiento: promedio de varianzas
pronosticadas
  # para los próximos 34 días hábiles (k=1..34), arrancando desde h1 =
h_{t+1}
  Tpath <- 34
  k <- 1:Tpath
  fvar_path <- ifelse(k == 1, fc$h1, fc$h_bar + (fc$pers^(k - 1)) * (fc$h1
- fc$h_bar))
  vol_garch_prom_34d[tk] <- sqrt(mean(fvar_path)) * sqrt(252)
}

# Volatilidad implícita TradingView (llenar, % anual)
vol_impl_tv <- c(NVDA = NA_real_, AMD = NA_real_, XOM = NA_real_)

razon <- vol_garch_prom_34d / vol_impl_tv
senal <- ifelse(is.na(razon), NA_character_,
                ifelse(razon >= 1.10, "GARCH > implícita (posible barata en
vol)",
                ifelse(razon <= 0.90, "GARCH < implícita (posible cara en
vol)",
                                      "Sin ventaja clara (|diff| < 10%)")))

tabla_vols <- data.frame(
  Accion = tickers,
  Vol_hist_1y_anual = as.numeric(vol_hist_1y_vec),
  Vol_EWMA_hoy_anual = as.numeric(vol_ewma_hoy),
  Vol_prom_GARCH_34d = as.numeric(vol_garch_prom_34d),
  Sigma implicita TV = as.numeric(vol impl tv),
  Razon GARCH sobre Impl = as.numeric(razon),
  Senal_10pct = as.character(senal),
  row.names = NULL
)

tabla_garch_plazos <- data.frame(
  Accion = tickers,
  Vol_GARCH_10d  = as.numeric(vol_garch_h[, "10d"]),
  Vol GARCH 30d  = as.numeric(vol garch h[, "30d"]),
  Vol GARCH 34d  = as.numeric(vol garch h[, "34d"]),
  Vol_GARCH_126d = as.numeric(vol_garch_h[, "126d"]),
  Vol_GARCH_252d = as.numeric(vol_garch_h[, "252d"]),
  row.names = NULL
)

print(tabla_vols)
print(tabla_garch_plazos)
```

BLOQUE 4 (corregido): Monte Carlo con GARCH, deriva neutral y VaR

```
## Bloque 4: Monte Carlo con GARCH(1,1) hasta vencimiento (100,000
trayectorias)
## Correcciones incluidas:
## - 34 días hábiles como horizonte de simulación
## - Deriva neutral al riesgo con corrección de Itô (sin media histórica)
## - Varianza inicial usa h_{t+1} (día siguiente)

set.seed(123)
N <- 100000
T_venc_bd <- 34      # días hábiles hasta vencimiento (aprox)
T_var_bd  <- 10      # VaR a 10 días hábiles
dt_year <- 1/252     # tamaño de paso en años (1 día hábil)

tickers <- c("NVDA", "AMD", "XOM")

# Spots de hoy
S0 <- as.numeric(datos[nrow(datos), tickers])
names(S0) <- tickers

# Dividend yield continuo por acción
q_yield <- c(NVDA = 0.0047, AMD = 0.0000, XOM = 0.0225)

# Tasa libre de riesgo continua
r <- 0.0394

# Simulación de precio con GARCH: log-return ~ N(m, h_dec) con m = (r-q)*dt
- 0.5*h_dec
# h pct está en %^2 (varianza diaria de rendimientos en %).
simula precio garch Q <- function(S0, x ret pct, fit, N, T venc bd,
T_var_bd, r, q, dt_year) {
  x_ret_pct <- as.numeric(x_ret_pct)
  x_ret_pct <- x_ret_pct[is.finite(x_ret_pct)]

  omega <- fit$omega
  alpha <- fit$alpha
  beta  <- fit$beta

  # Obtener h last y h next (día siguiente)
  tmp <- garch last h and next(x ret pct, fit)
  h_pct <- rep(tmp$h_next, N)      # arrancar en h_{t+1}

  S <- rep(S0, N)
  S10 <- rep(NA real , N)

  for (t in 1:T_venc_bd) {
    z <- rnorm(N)

    # Innovación en % para actualizar GARCH
    e_pct <- sqrt(h_pct) * z

    # Convertir a escala decimal para mover precio
    h_dec <- h_pct / 10000              # (%^2) -> (decimal^2)
    shock_dec <- e_pct / 100            # % -> decimal
```

```
    # Deriva neutral con Itô
    drift <- (r - q) * dt_year - 0.5 * h_dec
    r_dec <- drift + shock_dec

    S <- S * exp(r_dec)

    # Actualizar varianza condicional para el siguiente día hábil
    h_pct <- omega + alpha * (e_pct^2) + beta * h_pct

    if (t == T_var_bd) S10 <- S
  }

  p <- as.numeric(quantile(S, probs = c(0.05, 0.25, 0.50, 0.75, 0.95),
na.rm = TRUE))
  names(p) <- c("P5","P25","P50","P75","P95")

  prob_subir <- mean(S > S0)

  # VaR 99% a 10 días: pérdida relativa (S0 - S10)/S0
  loss10 <- (S0 - S10) / S0
  var99_10d <- as.numeric(quantile(loss10, probs = 0.99, na.rm = TRUE))

  list(P5 = p["P5"], P25 = p["P25"], P50 = p["P50"], P75 = p["P75"], P95 =
p["P95"],
       prob_subir = prob_subir, var99_10d = var99_10d)
}

res <- lapply(tickers, function(tk) {
  simula_precio_garch_Q(
    S0 = S0[tk],
    x_ret_pct = ret[[tk]],
    fit = ajustes[[tk]],
    N = N,
    T_venc_bd = T_venc_bd,
    T_var_bd = T_var_bd,
    r = r,
    q = q_yield[tk],
    dt_year = dt_year
  )
})
names(res) <- tickers

tabla_mc <- data.frame(
  Accion = tickers,
  Precio_hoy = as.numeric(S0[tickers]),
  P5  = sapply(res, `[[`, "P5"),
  P25 = sapply(res, `[[`, "P25"),
  P50 = sapply(res, `[[`, "P50"),
  P75 = sapply(res, `[[`, "P75"),
  P95 = sapply(res, `[[`, "P95"),
  Prob_de_subir = sapply(res, `[[`, "prob_subir"),
  VaR_99pct_10d = sapply(res, `[[`, "var99_10d"),
  row.names = NULL
)

print(tabla_mc)
```

BLOQUE 5 (corregido): Árbol binomial CRR americano con T = 49/365 y σ histórica 1 año con 252 returns

```
## Bloque 5: Árbol binomial CRR (americano y europeo) con dividend yield
continuo q
## Correcciones incluidas:
## - T = 49/365 (días naturales)
## - Volatilidad histórica 1 año usa últimos 252 rendimientos

# Parámetros de mercado
r <- 0.0394
T <- 49/365
Nsteps <- 500

q_yield <- c(NVDA = 0.0047, AMD = 0.0000, XOM = 0.0225)

inst <- data.frame(
  Accion = c("NVDA", "AMD", "XOM"),
  Tipo   = c("call", "put", "call"),
  K      = c(200, 500, 185),
  stringsAsFactors = FALSE
)

S0 <- as.numeric(datos[nrow(datos), inst$Accion])
names(S0) <- inst$Accion

# Volatilidad histórica 1 año (últimos 252 rendimientos), anualizada, en %
vol_hist_1y <- sapply(inst$Accion, function(tk) vol_hist_1y(ret[[tk]]))

# Volatilidad GARCH promedio esperada a 34 días hábiles (en % anual), del
Bloque 3
vol_garch_34 <- vol_garch_prom_34d[inst$Accion]

# Entradas: TradingView e instrumentos de mercado (llenar)
# Vol implícita TradingView en % anual
# (si viene en decimal, multiplica por 100 antes de ponerla aquí)
vol_impl_tv <- c(NVDA = NA_real_, AMD = NA_real_, XOM = NA_real_)

# Bid y Ask del mercado para cada opción del listado (USD por acción)
bid_mkt <- c(NVDA = NA_real_, AMD = NA_real_, XOM = NA_real_)
ask_mkt <- c(NVDA = NA_real_, AMD = NA_real_, XOM = NA_real_)

crr_price <- function(S0, K, r, q, sigma_ann, T, Nsteps, type =
c("call","put"), american = TRUE) {
  type <- match.arg(type)

  sigma <- sigma_ann / 100
  dt <- T / Nsteps
  u <- exp(sigma * sqrt(dt))
  d <- 1 / u

  disc <- exp(-r * dt)
```

```
  p <- (exp((r - q) * dt) - d) / (u - d)
  if (!is.finite(p) || p < 0 || p > 1) return(NA_real_)

  j <- 0:Nsteps
  ST <- S0 * (u^j) * (d^(Nsteps - j))

  if (type == "call") V <- pmax(ST - K, 0) else V <- pmax(K - ST, 0)

  for (i in (Nsteps - 1):0) {
    V <- disc * (p * V[2:(i + 2)] + (1 - p) * V[1:(i + 1)])

    if (american) {
      j <- 0:i
      Sij <- S0 * (u^j) * (d^(i - j))
      if (type == "call") intrinsic <- pmax(Sij - K, 0) else intrinsic <-
pmax(K - Sij, 0)
      V <- pmax(V, intrinsic)
    }
  }

  V[1]
}

valua_instrumento <- function(tk, tipo, K, r, q, T, Nsteps,
                              sigma_hist, sigma_garch, sigma_impl, bid,
ask) {
  sigmas <- c(Historica_1y = sigma_hist, GARCH_34d = sigma_garch,
Implicita_TV = sigma_impl)

  out <- lapply(names(sigmas), function(nm) {
    sig <- sigmas[[nm]]

    price_eur <- crr_price(S0 = S0[tk], K = K, r = r, q = q, sigma_ann =
sig,
                           T = T, Nsteps = Nsteps, type = tipo, american =
FALSE)
    price ame <- crr price(S0 = S0[tk], K = K, r = r, q = q, sigma ann =
sig,
                           T = T, Nsteps = Nsteps, type = tipo, american =
TRUE)

    data.frame(
      Accion = tk,
      Tipo = tipo,
      K = K,
      Sigma fuente = nm,
      Sigma_anual_pct = sig,
      Precio_Europeo = price_eur,
      Precio_Americano = price_ame,
      Prima ejercicio anticipado = price ame - price eur,
      Bid = bid,
      Ask = ask,
      row.names = NULL,
      stringsAsFactors = FALSE
    )
  })

  do.call(rbind, out)
}
```

```
tabla_arbol <- do.call(rbind, lapply(1:nrow(inst), function(i) {
  tk <- inst$Accion[i]
  valua instrumento(
    tk = tk,
    tipo = inst$Tipo[i],
    K = inst$K[i],
    r = r,
    q = q yield[tk],
    T = T,
    Nsteps = Nsteps,
    sigma_hist = vol_hist_1y[tk],
    sigma_garch = vol_garch_34[tk],
    sigma impl = vol impl tv[tk],
    bid = bid_mkt[tk],
    ask = ask_mkt[tk]
  )
}))

print(tabla_arbol)
```

BLOQUE 6 (corregido): Griegas y nodos; rho por +1 pp

```
## Bloque 6: Griegas desde el árbol CRR y nodos con decisión
## Correcciones incluidas:
## - Usa T = 49/365 (ya definido en Bloque 5)
## - Rho expresado por +1 punto porcentual (1 pp), no por 1.00 de tasa

pos <- data.frame(
  Accion = c("NVDA", "AMD", "XOM"),
  Tipo   = c("call", "put", "call"),
  K      = c(200, 500, 185),
  Contratos = c(2, 1, 3),
  stringsAsFactors = FALSE
)
mult <- 100

bump_sigma_pto <- 1.0   # +1 punto de vol
bump_r_pp <- 0.01       # +1 punto porcentual en tasa (decimal)

crr_american_with_nodes <- function(S0, K, r, q, sigma_ann, T, Nsteps, type
= c("call","put"),
                                    keep_steps = c(1,2,4)) {
  type <- match.arg(type)

  sigma <- sigma_ann / 100
  dt <- T / Nsteps
  u <- exp(sigma * sqrt(dt))
  d <- 1 / u
  disc <- exp(-r * dt)
  p <- (exp((r - q) * dt) - d) / (u - d)

  if (!is.finite(p) || p < 0 || p > 1) return(list(price = NA_real_, nodes
= NULL, dt = dt))
```

```
  j <- 0:Nsteps
  ST <- S0 * (u^j) * (d^(Nsteps - j))
  if (type == "call") V <- pmax(ST - K, 0) else V <- pmax(K - ST, 0)

  nodes <- list()

  for (i in (Nsteps - 1):0) {
    V_cont <- disc * (p * V[2:(i + 2)] + (1 - p) * V[1:(i + 1)])

    j <- 0:i
    Sij <- S0 * (u^j) * (d^(i - j))
    if (type == "call") intrinsic <- pmax(Sij - K, 0) else intrinsic <-
pmax(K - Sij, 0)

    exercise <- intrinsic > V_cont
    V <- pmax(V_cont, intrinsic)

    if (i %in% keep_steps) {
      nodes[[paste0("step_", i)]] <- data.frame(
        step = i,
        j = j,
        S = Sij,
        Continuar = V_cont,
        Intrinseco = intrinsic,
        Valor_americano = V,
        Decision = ifelse(exercise, "EJERCER", "ESPERAR"),
        row.names = NULL
      )
    }
  }

  list(price = V[1], nodes = nodes, dt = dt)
}

crr_greeks <- function(S0, K, r, q, sigma_ann, T, Nsteps, type =
c("call","put")) {
  type <- match.arg(type)

  out <- crr_american_with_nodes(S0, K, r, q, sigma_ann, T, Nsteps, type,
keep_steps = c(1,2))
  if (!is.finite(out$price)) return(list(delta = NA, gamma = NA, theta =
NA, vega = NA, rho_1pp = NA))

  dt <- out$dt
  n1 <- out$nodes[["step_1"]]
  n2 <- out$nodes[["step_2"]]

  # Delta desde paso 1
  Vd <- n1$Valor_americano[n1$j == 0]
  Vu <- n1$Valor_americano[n1$j == 1]
  Sd <- n1$S[n1$j == 0]
  Su <- n1$S[n1$j == 1]
  delta <- (Vu - Vd) / (Su - Sd)

  # Gamma desde paso 2
  Vdd <- n2$Valor_americano[n2$j == 0]
  Vud <- n2$Valor_americano[n2$j == 1]
  Vuu <- n2$Valor_americano[n2$j == 2]
```

```
  Sdd <- n2$S[n2$j == 0]
  Sud <- n2$S[n2$j == 1]
  Suu <- n2$S[n2$j == 2]

  delta_up   <- (Vuu - Vud) / (Suu - Sud)
  delta_down <- (Vud - Vdd) / (Sud - Sdd)
  gamma <- (delta_up - delta_down) / ((Suu - Sdd) / 2)

  # Theta aproximada (USD por día natural)
  V0 <- out$price
  theta_per_year <- (Vud - V0) / (2 * dt)
  theta_per_day <- theta_per_year / 365

  # Vega por 1 punto de vol
  price_up <- crr_price(S0, K, r, q, sigma_ann + bump_sigma_pto, T, Nsteps,
type = type, american = TRUE)
  price_dn <- crr_price(S0, K, r, q, sigma_ann - bump_sigma_pto, T, Nsteps,
type = type, american = TRUE)
  vega <- (price_up - price_dn) / (2 * bump_sigma_pto)

  # Rho por +1 punto porcentual (1 pp): (V(r+0.01) - V(r-0.01))/2
  price_ru <- crr_price(S0, K, r + bump_r_pp, q, sigma_ann, T, Nsteps, type
= type, american = TRUE)
  price_rd <- crr_price(S0, K, r - bump_r_pp, q, sigma_ann, T, Nsteps, type
= type, american = TRUE)
  rho_1pp <- (price_ru - price_rd) / 2

  list(delta = delta, gamma = gamma, theta = theta_per_day, vega = vega,
rho_1pp = rho_1pp)
}

# Griegas usando sigma implícita TradingView como base
tabla_greeks <- do.call(rbind, lapply(1:nrow(pos), function(i) {
  tk <- pos$Accion[i]
  tipo <- pos$Tipo[i]
  K <- pos$K[i]
  contratos <- pos$Contratos[i]

  sigma_use <- vol_impl_tv[tk]

  g <- crr_greeks(S0 = S0[tk], K = K, r = r, q = q_yield[tk],
                  sigma_ann = sigma_use, T = T, Nsteps = Nsteps, type =
tipo)

  data.frame(
    Accion = tk,
    Tipo = tipo,
    K = K,
    Sigma_usada_pct = sigma_use,
    Contratos = contratos,
    Delta_opcion = g$delta,
    Gamma_opcion = g$gamma,
    Theta_opcion_USD_dia = g$theta,
    Vega_opcion_USD_por_1pto = g$vega,
    Rho_opcion_USD_por_1pp = g$rho_1pp,
    Delta_posicion_acciones = g$delta * contratos * mult,
    Gamma_posicion = g$gamma * contratos * mult,
    Theta_posicion_USD_dia = g$theta * contratos * mult,
    Vega_posicion_USD_por_1pto = g$vega * contratos * mult,
    Rho_posicion_USD_por_1pp = g$rho_1pp * contratos * mult,
```

```
    row.names = NULL
  )
}))

print(tabla_greeks)

# Nodos de los primeros 2 pasos y del paso 4 con decisión ejercer o esperar
for (i in 1:nrow(pos)) {
  tk <- pos$Accion[i]
  tipo <- pos$Tipo[i]
  K <- pos$K[i]
  sigma_use <- vol_impl_tv[tk]

  cat("\n========================\n")
  cat("Nodos y decisión:", tk, tipo, "K =", K, "sigma =", sigma_use, "%\n")
  cat("========================\n")

  dbg <- crr american with nodes(S0 = S0[tk], K = K, r = r, q =
q_yield[tk],
                                 sigma_ann = sigma_use, T = T, Nsteps =
Nsteps,
                                 type = tipo, keep_steps = c(1,2,4))

  print(dbg$nodes[["step_1"]])
  print(dbg$nodes[["step_2"]])
  print(dbg$nodes[["step_4"]])
}
```

BLOQUE 7 (corregido): Señal, straddle y rentabilidad a 2 semanas con revaluación y T remanente correcto

```
## Bloque 7: Señal de volatilidad, straddle y rentabilidad a 2 semanas
## Correcciones incluidas:
## - Señal usa vol GARCH prom 34d / vol impl tv con umbral 10%
## - Straddle: costo y break-even sin cambio
## - Simulación 10 días hábiles con deriva neutral e Itô
## - Tiempo remanente a 10 días hábiles: (49 - 14)/365

tickers <- c("NVDA", "AMD", "XOM")

# Tiempo total en años (días naturales)
T_total_days_nat <- 49
T_total <- T_total_days_nat / 365

# Dos semanas: 10 días hábiles ~ 14 días naturales
T_2w_bd <- 10
T_2w_nat_equiv <- 14
T_rem <- (T_total_days_nat - T_2w_nat_equiv) / 365

# Simulación
Npaths <- 100000
set.seed(123)
```

```
dt_year <- 1/252

# Spots de hoy
S0_all <- as.numeric(datos[nrow(datos), tickers])
names(S0_all) <- tickers

# 1) Señal de volatilidad
razon vol <- vol garch prom 34d[tickers] / vol impl tv[tickers]
senal_vol <- ifelse(is.na(razon_vol), NA_character_,
                    ifelse(razon_vol >= 1.10, "POSIBLE BARATA en vol (GARCH
> implícita)",
                    ifelse(razon_vol <= 0.90, "POSIBLE CARA en vol (GARCH <
implícita)",
                                              "SIN VENTAJA clara (|diff| <
10%)")))

tabla_senal <- data.frame(
  Accion = tickers,
  Sigma_GARCH_34d = as.numeric(vol_garch_prom_34d[tickers]),
  Sigma_implicita_TV = as.numeric(vol_impl_tv[tickers]),
  Razon = as.numeric(razon_vol),
  Senal_10pct = as.character(senal_vol),
  row.names = NULL
)
print(tabla_senal)

# 2) Straddle: costo y puntos de equilibrio (call + put al mismo K)
K_map <- c(NVDA = 200, AMD = 500, XOM = 185)

# Bid y ask para CALL y PUT del straddle (llenar, USD por acción)
bid_call <- c(NVDA = NA_real_, AMD = NA_real_, XOM = NA_real_)
ask_call <- c(NVDA = NA_real_, AMD = NA_real_, XOM = NA_real_)
bid_put  <- c(NVDA = NA_real_, AMD = NA_real_, XOM = NA_real_)
ask_put  <- c(NVDA = NA_real_, AMD = NA_real_, XOM = NA_real_)

costo straddle <- ask call + ask put
eq inf <- S0 all - costo straddle
eq_sup <- S0_all + costo_straddle

tabla_straddle <- data.frame(
  Accion = tickers,
  K = as.numeric(K_map[tickers]),
  Spot_hoy = as.numeric(S0_all[tickers]),
  Costo_straddle_ask = as.numeric(costo_straddle[tickers]),
  Equilibrio inferior = as.numeric(eq inf[tickers]),
  Equilibrio superior = as.numeric(eq sup[tickers]),
  row.names = NULL
)
print(tabla_straddle)

# 3) Simular S 10 con GARCH bajo medida neutral (mismo motor que Bloque 4)
simula_S10_garch_Q <- function(S0, x_ret_pct, fit, N, T_var_bd, r, q,
dt_year) {
  x_ret_pct <- as.numeric(x_ret_pct)
  x ret pct <- x ret pct[is.finite(x ret pct)]

  omega <- fit$omega
  alpha <- fit$alpha
  beta  <- fit$beta
```

```r
  tmp <- garch_last_h_and_next(x_ret_pct, fit)
  h_pct <- rep(tmp$h_next, N)

  S <- rep(S0, N)

  for (t in 1:T_var_bd) {
    z <- rnorm(N)

    e_pct <- sqrt(h_pct) * z
    h_dec <- h_pct / 10000
    shock_dec <- e_pct / 100

    drift <- (r - q) * dt_year - 0.5 * h_dec
    r_dec <- drift + shock_dec

    S <- S * exp(r_dec)

    h_pct <- omega + alpha * (e_pct^2) + beta * h_pct
  }

  S
}

# 4) Revaluación con árbol a tiempo remanente T_rem
Nsteps_reval <- 500

revalua_opcion_arbol <- function(S_now, K, tipo, sigma_ann, r, q, T_rem,
Nsteps) {
  crr_price(S0 = S_now, K = K, r = r, q = q, sigma_ann = sigma_ann,
            T = T_rem, Nsteps = Nsteps, type = tipo, american = TRUE)
}

# Posiciones
pos <- data.frame(
  Accion = c("NVDA", "AMD", "XOM"),
  Tipo = c("call", "put", "call"),
  K = c(200, 500, 185),
  Contratos = c(2, 1, 3),
  stringsAsFactors = FALSE
)
mult <- 100

# Ask pagado por cada posición (llenar, USD por acción)
ask_pos <- c(NVDA = NA_real_, AMD = NA_real_, XOM = NA_real_)

res_2w <- lapply(1:nrow(pos), function(i) {
  tk <- pos$Accion[i]
  tipo <- pos$Tipo[i]
  K <- pos$K[i]
  contratos <- pos$Contratos[i]

  S10 <- simula_S10_garch_Q(
    S0 = S0_all[tk],
    x_ret_pct = ret[[tk]],
    fit = ajustes[[tk]],
    N = Npaths,
    T_var_bd = T_2w_bd,
    r = r,
    q = q_yield[tk],
    dt_year = dt_year
```

```
  )

  sigma_use <- vol_impl_tv[tk]
  q_use <- q_yield[tk]

  V10 <- vapply(S10, function(s) {
    revalua_opcion_arbol(S_now = s, K = K, tipo = tipo, sigma_ann =
sigma_use,
                         r = r, q = q_use, T_rem = T_rem, Nsteps =
Nsteps_reval)
  }, numeric(1))

  precio_pagado <- ask_pos[tk]
  pnl_por_accion <- V10 - precio_pagado
  pnl_pos <- pnl_por_accion * contratos * mult

  data.frame(
    Accion = tk,
    Tipo = tipo,
    K = K,
    Contratos = contratos,
    T_rem_anios = T_rem,
    Costo_total_USD = precio_pagado * contratos * mult,
    PnL_esperado_2sem_USD = mean(pnl_pos, na.rm = TRUE),
    Prob_de_ganar = mean(pnl_pos > 0, na.rm = TRUE),
    P5_USD = as.numeric(quantile(pnl_pos, probs = 0.05, na.rm = TRUE)),
    P95_USD = as.numeric(quantile(pnl_pos, probs = 0.95, na.rm = TRUE)),
    row.names = NULL
  )
})

tabla_2w <- do.call(rbind, res_2w)
print(tabla_2w)
```

Si me pegas los tres valores de `vol_impl_tv` y los `ask_pos` (y si quieres también `ask_call/ask_put` para straddles), te puedo decir si las salidas ya quedan numéricamente alineadas con su PDF.

[DONE]

2/11/2026 - 7:58:00 p.m.

Gracias, ya revisé las correcciones y los resultados coinciden con los nuestros. Haz tres ajustes finales: (1) en el Bloque 5 renombra el vector vol_hist_1y a vol_hist_1y_vec para no pisar la función del Bloque 3; (2) quita los sufijos de nombre en la columna Sigma_fuente con unname(); (3) junta los Bloques 1 a 7 en un solo archivo R Markdown ordenado, con cada bloque en su propio chunk de R, títulos en Markdown fuera del código, y Npaths = 5000 en el Bloque 7 con un comentario indicando que se reduce por tiempo de cómputo. No uses guiones largos.



```
---
title: "Tarea Clase 6: GARCH, Monte Carlo y Árboles Binomiales (NVDA, AMD, XOM)"
author: "Equipo"
date: "`r format(Sys.Date())`"
output: html_document
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE, warning = FALSE, message = FALSE)
```

Bloque 1. Lectura de datos, rendimientos y volatilidades recientes

```
## R base

# Leer CSV (ajusta el nombre si aplica)
datos <- read.csv("precios clase6.csv", stringsAsFactors = FALSE)

# Convertir y ordenar por fecha
datos$Date <- as.Date(datos$Date)
datos <- datos[order(datos$Date), ]

# Extraer precios
precios <- datos[, c("NVDA", "AMD", "XOM")]
precios <- as.data.frame(lapply(precios, as.numeric))

# Rendimientos logarítmicos diarios en porcentaje
ret <- as.data.frame(lapply(precios, function(p) 100 * diff(log(p))))
ret$Date <- datos$Date[-1]

# Volatilidad anualizada
vol_anual <- function(x) sd(x, na.rm = TRUE) * sqrt(252)

n <- nrow(ret)
ult30 <- max(1, n - 30 + 1):n
ulty  <- max(1, n - 252 + 1):n

vol30 <- sapply(ret[, c("NVDA","AMD","XOM")], function(x)
vol_anual(x[ult30]))
vol1y <- sapply(ret[, c("NVDA","AMD","XOM")], function(x)
vol anual(x[ulty]))

tabla_vol <- data.frame(
  Accion = c("NVDA", "AMD", "XOM"),
  Vol 30d anual = as.numeric(vol30),
  Vol 1y anual  = as.numeric(vol1y),
  row.names = NULL
)

print(tabla_vol)
```
```

Bloque 2. Estimación GARCH(1,1) por MLE (innovaciones normales)

```
## R base, con optim y restricciones por cotas + penalización para
alpha+beta<1

fit_garch11_norm <- function(x) {
  x <- as.numeric(x)
  x <- x[is.finite(x)]
  mu <- mean(x)
  e  <- x - mu
  n  <- length(e)

  h0 <- var(e)
  if (!is.finite(h0) || h0 <= 0) h0 <- 1

  nll <- function(par) {
    omega <- par[1]
    alpha <- par[2]
    beta  <- par[3]

    if (omega <= 0 || alpha < 0 || beta < 0) return(1e12)
    if (alpha + beta >= 0.999999) return(1e12 + 1e8 * (alpha + beta -
0.999999)^2)

    h <- numeric(n)
    h[1] <- h0
    for (t in 2:n) {
      h[t] <- omega + alpha * e[t - 1]^2 + beta * h[t - 1]
      if (!is.finite(h[t]) || h[t] <= 0) return(1e12)
    }

    ll <- -0.5 * (log(2 * pi) + log(h) + (e^2) / h)
    -sum(ll)
  }

  alpha0 <- 0.05
  beta0  <- 0.90
  omega0 <- max(1e-6, h0 * (1 - alpha0 - beta0))

  opt <- optim(
    par    = c(omega0, alpha0, beta0),
    fn     = nll,
    method = "L-BFGS-B",
    lower  = c(1e-12, 0, 0),
    upper  = c(Inf, 1, 1)
  )

  par <- opt$par
  omega <- par[1]; alpha <- par[2]; beta <- par[3]
  pers  <- alpha + beta

  # Reconstrucción de h_t para "volatilidad de hoy"
  h <- numeric(n)
  h[1] <- h0
  for (t in 2:n) h[t] <- omega + alpha * e[t - 1]^2 + beta * h[t - 1]
```

```
  h_lr <- omega / (1 - pers)
  vol_lr_ann    <- sqrt(h_lr) * sqrt(252)
  vol_today_ann <- sqrt(h[n]) * sqrt(252)

  half_life <- if (pers > 0 && pers < 1) log(0.5) / log(pers) else NA_real_

  list(
    omega = omega, alpha = alpha, beta = beta,
    persistencia = pers,
    vol_lr_ann = vol_lr_ann,
    vol_today_ann = vol_today_ann,
    half_life = half_life,
    converged = (opt$convergence == 0)
  )
}

tickers <- c("NVDA", "AMD", "XOM")
ajustes <- lapply(tickers, function(tk) fit_garch11_norm(ret[[tk]]))
names(ajustes) <- tickers

tabla_garch <- data.frame(
  Accion = tickers,
  omega = sapply(ajustes, `[[`, "omega"),
  alpha = sapply(ajustes, `[[`, "alpha"),
  beta  = sapply(ajustes, `[[`, "beta"),
  Persistencia = sapply(ajustes, `[[`, "persistencia"),
  Vol_largo_plazo_anual = sapply(ajustes, `[[`, "vol_lr_ann"),
  Vol_hoy_anual = sapply(ajustes, `[[`, "vol_today_ann"),
  Vida_media_shock_dias = sapply(ajustes, `[[`, "half_life"),
  Convergio = sapply(ajustes, `[[`, "converged"),
  row.names = NULL
)

print(tabla_garch)
```

Bloque 3. EWMA y pronóstico de varianza GARCH (reversión a la media)

```
tickers <- c("NVDA", "AMD", "XOM")
lambda <- 0.94

# EWMA de hoy (anualizada) en % usando rendimientos en % diario
ewma_vol_hoy <- function(x, lambda = 0.94) {
  x <- as.numeric(x)
  x <- x[is.finite(x)]
  n <- length(x)
  if (n < 2) return(NA_real_)

  e <- x - mean(x)
  s2 <- var(e)
  if (!is.finite(s2) || s2 <= 0) s2 <- 1

  for (t in 2:n) s2 <- lambda * s2 + (1 - lambda) * e[t - 1]^2
  sqrt(s2) * sqrt(252)
}
```

```r
# Histórica 1 año: últimos 252 rendimientos, anualizada, en %
vol_hist_1y <- function(x) {
  x <- as.numeric(x)
  x <- x[is.finite(x)]
  n <- length(x)
  idx <- max(1, n - 252 + 1):n
  sd(x[idx], na.rm = TRUE) * sqrt(252)
}

vol_ewma_hoy <- sapply(tickers, function(tk) ewma_vol_hoy(ret[[tk]],
lambda))
vol_hist_1y_vec <- sapply(tickers, function(tk) vol_hist_1y(ret[[tk]]))

# h_t (última) y h_{t+1} (día siguiente) para arrancar pronósticos y
simulación
garch_last_h_and_next <- function(x, fit) {
  x <- as.numeric(x)
  x <- x[is.finite(x)]
  mu <- mean(x)
  e  <- x - mu
  n  <- length(e)

  omega <- fit$omega
  alpha <- fit$alpha
  beta  <- fit$beta

  h0 <- var(e)
  if (!is.finite(h0) || h0 <= 0) h0 <- 1

  h <- numeric(n)
  h[1] <- h0
  for (t in 2:n) h[t] <- omega + alpha * e[t - 1]^2 + beta * h[t - 1]

  h_last <- h[n]
  e_last <- e[n]
  h_next <- omega + alpha * e_last^2 + beta * h_last

  list(h_last = h_last, h_next = h_next)
}

# Pronóstico por reversión a la media arrancando en h_{t+1}
garch_forecast_var_from_next <- function(x, fit, horizons = c(10, 30, 34,
126, 252)) {
  tmp <- garch_last_h_and_next(x, fit)

  omega <- fit$omega
  alpha <- fit$alpha
  beta  <- fit$beta
  pers  <- alpha + beta

  h_bar <- omega / (1 - pers)
  h1 <- tmp$h_next

  fvar <- sapply(horizons, function(H) {
    if (H <= 1) return(h1)
    h_bar + (pers^(H - 1)) * (h1 - h_bar)
  })

  list(h1 = h1, h_bar = h_bar, pers = pers, fvar = fvar)
```

```
}

horizons <- c(10, 30, 34, 126, 252)

vol_garch_h <- matrix(NA_real_, nrow = length(tickers), ncol =
length(horizons),
                      dimnames = list(tickers, paste0(horizons, "d")))
vol garch prom 34d <- setNames(rep(NA real , length(tickers)), tickers)

for (tk in tickers) {
  fc <- garch_forecast_var_from_next(ret[[tk]], ajustes[[tk]], horizons)

  vol garch h[tk, ] <- sqrt(fc$fvar) * sqrt(252)

  # Volatilidad promedio esperada hasta vencimiento en 34 días hábiles
  Tpath <- 34
  k <- 1:Tpath
  fvar path <- ifelse(k == 1, fc$h1, fc$h bar + (fc$pers^(k - 1)) * (fc$h1
- fc$h_bar))
  vol_garch_prom_34d[tk] <- sqrt(mean(fvar_path)) * sqrt(252)
}

# Volatilidad implícita TradingView (llenar, % anual)
vol_impl_tv <- c(NVDA = NA_real_, AMD = NA_real_, XOM = NA_real_)

razon <- vol_garch_prom_34d / vol_impl_tv
senal <- ifelse(is.na(razon), NA_character_,
                ifelse(razon >= 1.10, "GARCH > implícita (posible barata en
vol)",
                ifelse(razon <= 0.90, "GARCH < implícita (posible cara en
vol)",
                                      "Sin ventaja clara (|diff| < 10%)")))

tabla_vols <- data.frame(
  Accion = tickers,
  Vol hist 1y anual = as.numeric(vol hist 1y vec),
  Vol EWMA hoy anual = as.numeric(vol ewma hoy),
  Vol_prom_GARCH_34d = as.numeric(vol_garch_prom_34d),
  Sigma_implicita_TV = as.numeric(vol_impl_tv),
  Razon_GARCH_sobre_Impl = as.numeric(razon),
  Senal 10pct = as.character(senal),
  row.names = NULL
)

tabla garch plazos <- data.frame(
  Accion = tickers,
  Vol_GARCH_10d  = as.numeric(vol_garch_h[, "10d"]),
  Vol_GARCH_30d  = as.numeric(vol_garch_h[, "30d"]),
  Vol_GARCH_34d  = as.numeric(vol_garch_h[, "34d"]),
  Vol GARCH 126d = as.numeric(vol garch h[, "126d"]),
  Vol GARCH 252d = as.numeric(vol garch h[, "252d"]),
  row.names = NULL
)

print(tabla vols)
print(tabla_garch_plazos)
```

Bloque 4. Monte Carlo con GARCH bajo medida neutral al riesgo (hasta vencimiento)

```
set.seed(123)
N <- 100000
T_venc_bd <- 34
T_var_bd  <- 10
dt_year <- 1/252

tickers <- c("NVDA", "AMD", "XOM")

S0 <- as.numeric(datos[nrow(datos), tickers])
names(S0) <- tickers

q_yield <- c(NVDA = 0.0047, AMD = 0.0000, XOM = 0.0225)
r <- 0.0394

simula_precio_garch_Q <- function(S0, x_ret_pct, fit, N, T_venc_bd,
T_var_bd, r, q, dt_year) {
  x_ret_pct <- as.numeric(x_ret_pct)
  x_ret_pct <- x_ret_pct[is.finite(x_ret_pct)]

  omega <- fit$omega
  alpha <- fit$alpha
  beta  <- fit$beta

  tmp <- garch_last_h_and_next(x_ret_pct, fit)
  h_pct <- rep(tmp$h_next, N)

  S <- rep(S0, N)
  S10 <- rep(NA_real_, N)

  for (t in 1:T_venc_bd) {
    z <- rnorm(N)

    e_pct <- sqrt(h_pct) * z
    h_dec <- h_pct / 10000
    shock_dec <- e_pct / 100

    drift <- (r - q) * dt_year - 0.5 * h_dec
    r_dec <- drift + shock_dec

    S <- S * exp(r_dec)

    h_pct <- omega + alpha * (e_pct^2) + beta * h_pct

    if (t == T_var_bd) S10 <- S
  }

  p <- as.numeric(quantile(S, probs = c(0.05, 0.25, 0.50, 0.75, 0.95),
na.rm = TRUE))
  names(p) <- c("P5","P25","P50","P75","P95")
```

```
  prob_subir <- mean(S > S0)

  loss10 <- (S0 - S10) / S0
  var99_10d <- as.numeric(quantile(loss10, probs = 0.99, na.rm = TRUE))

  list(P5 = p["P5"], P25 = p["P25"], P50 = p["P50"], P75 = p["P75"], P95 =
p["P95"],
       prob subir = prob subir, var99 10d = var99 10d)
}

res <- lapply(tickers, function(tk) {
  simula_precio_garch_Q(
    S0 = S0[tk],
    x_ret_pct = ret[[tk]],
    fit = ajustes[[tk]],
    N = N,
    T_venc_bd = T_venc_bd,
    T var bd = T var bd,
    r = r,
    q = q_yield[tk],
    dt_year = dt_year
  )
})
names(res) <- tickers

tabla_mc <- data.frame(
  Accion = tickers,
  Precio_hoy = as.numeric(S0[tickers]),
  P5  = sapply(res, `[[`, "P5"),
  P25 = sapply(res, `[[`, "P25"),
  P50 = sapply(res, `[[`, "P50"),
  P75 = sapply(res, `[[`, "P75"),
  P95 = sapply(res, `[[`, "P95"),
  Prob_de_subir = sapply(res, `[[`, "prob_subir"),
  VaR_99pct_10d = sapply(res, `[[`, "var99_10d"),
  row.names = NULL
)

print(tabla_mc)
```

Bloque 5. Árbol binomial CRR americano (500 pasos) con dividend yield continuo

```
r <- 0.0394
T <- 49/365
Nsteps <- 500

q_yield <- c(NVDA = 0.0047, AMD = 0.0000, XOM = 0.0225)

inst <- data.frame(
  Accion = c("NVDA", "AMD", "XOM"),
  Tipo   = c("call", "put", "call"),
  K      = c(200, 500, 185),
```

```
  stringsAsFactors = FALSE
)

S0 <- as.numeric(datos[nrow(datos), inst$Accion])
names(S0) <- inst$Accion

# Ajuste (1): renombrar vector para no pisar la función vol_hist_1y del
Bloque 3
vol_hist_1y_vec <- sapply(inst$Accion, function(tk) vol_hist_1y(ret[[tk]]))
vol_garch_34 <- vol_garch_prom_34d[inst$Accion]

# Entradas a llenar
vol impl tv <- c(NVDA = NA real , AMD = NA real , XOM = NA real )
bid_mkt <- c(NVDA = NA_real_, AMD = NA_real_, XOM = NA_real_)
ask_mkt <- c(NVDA = NA_real_, AMD = NA_real_, XOM = NA_real_)

crr_price <- function(S0, K, r, q, sigma_ann, T, Nsteps, type =
c("call","put"), american = TRUE) {
  type <- match.arg(type)

  sigma <- sigma_ann / 100
  dt <- T / Nsteps
  u <- exp(sigma * sqrt(dt))
  d <- 1 / u

  disc <- exp(-r * dt)

  p <- (exp((r - q) * dt) - d) / (u - d)
  if (!is.finite(p) || p < 0 || p > 1) return(NA_real_)

  j <- 0:Nsteps
  ST <- S0 * (u^j) * (d^(Nsteps - j))

  if (type == "call") V <- pmax(ST - K, 0) else V <- pmax(K - ST, 0)

  for (i in (Nsteps - 1):0) {
    V <- disc * (p * V[2:(i + 2)] + (1 - p) * V[1:(i + 1)])

    if (american) {
      j <- 0:i
      Sij <- S0 * (u^j) * (d^(i - j))
      if (type == "call") intrinsic <- pmax(Sij - K, 0) else intrinsic <-
pmax(K - Sij, 0)
      V <- pmax(V, intrinsic)
    }
  }

  V[1]
}

valua instrumento <- function(tk, tipo, K, r, q, T, Nsteps,
                              sigma_hist, sigma_garch, sigma_impl, bid,
ask) {
  sigmas <- c(Historica_1y = sigma_hist, GARCH_34d = sigma_garch,
Implicita TV = sigma impl)

  out <- lapply(names(sigmas), function(nm) {
    sig <- sigmas[[nm]]
```

```
    price_eur <- crr_price(S0 = S0[tk], K = K, r = r, q = q, sigma_ann =
sig,
                           T = T, Nsteps = Nsteps, type = tipo, american =
FALSE)
    price_ame <- crr_price(S0 = S0[tk], K = K, r = r, q = q, sigma_ann =
sig,
                           T = T, Nsteps = Nsteps, type = tipo, american =
TRUE)

    data.frame(
      Accion = tk,
      Tipo = tipo,
      K = K,
      # Ajuste (2): quitar sufijos de nombres en Sigma_fuente
      Sigma_fuente = unname(nm),
      Sigma_anual_pct = sig,
      Precio_Europeo = price_eur,
      Precio Americano = price ame,
      Prima_ejercicio_anticipado = price_ame - price_eur,
      Bid = bid,
      Ask = ask,
      row.names = NULL,
      stringsAsFactors = FALSE
    )
  })

  do.call(rbind, out)
}

tabla_arbol <- do.call(rbind, lapply(1:nrow(inst), function(i) {
  tk <- inst$Accion[i]
  valua_instrumento(
    tk = tk,
    tipo = inst$Tipo[i],
    K = inst$K[i],
    r = r,
    q = q yield[tk],
    T = T,
    Nsteps = Nsteps,
    sigma_hist = vol_hist_1y_vec[tk],
    sigma garch = vol garch 34[tk],
    sigma_impl = vol_impl_tv[tk],
    bid = bid_mkt[tk],
    ask = ask_mkt[tk]
  )
}))

print(tabla_arbol)
```

Bloque 6. Griegas y nodos con decisión (ejercer o esperar)

```
pos <- data.frame(
  Accion = c("NVDA", "AMD", "XOM"),
  Tipo   = c("call", "put", "call"),
  K      = c(200, 500, 185),
```

```
  Contratos = c(2, 1, 3),
  stringsAsFactors = FALSE
)
mult <- 100

bump_sigma_pto <- 1.0
bump_r_pp <- 0.01

crr_american_with_nodes <- function(S0, K, r, q, sigma_ann, T, Nsteps, type
= c("call","put"),
                                    keep_steps = c(1,2,4)) {
  type <- match.arg(type)

  sigma <- sigma_ann / 100
  dt <- T / Nsteps
  u <- exp(sigma * sqrt(dt))
  d <- 1 / u
  disc <- exp(-r * dt)
  p <- (exp((r - q) * dt) - d) / (u - d)

  if (!is.finite(p) || p < 0 || p > 1) return(list(price = NA_real_, nodes
= NULL, dt = dt))

  j <- 0:Nsteps
  ST <- S0 * (u^j) * (d^(Nsteps - j))
  if (type == "call") V <- pmax(ST - K, 0) else V <- pmax(K - ST, 0)

  nodes <- list()

  for (i in (Nsteps - 1):0) {
    V_cont <- disc * (p * V[2:(i + 2)] + (1 - p) * V[1:(i + 1)])

    j <- 0:i
    Sij <- S0 * (u^j) * (d^(i - j))
    if (type == "call") intrinsic <- pmax(Sij - K, 0) else intrinsic <-
pmax(K - Sij, 0)

    exercise <- intrinsic > V_cont
    V <- pmax(V_cont, intrinsic)

    if (i %in% keep_steps) {
      nodes[[paste0("step_", i)]] <- data.frame(
        step = i,
        j = j,
        S = Sij,
        Continuar = V_cont,
        Intrinseco = intrinsic,
        Valor_americano = V,
        Decision = ifelse(exercise, "EJERCER", "ESPERAR"),
        row.names = NULL
      )
    }
  }

  list(price = V[1], nodes = nodes, dt = dt)
}

crr_greeks <- function(S0, K, r, q, sigma_ann, T, Nsteps, type =
c("call","put")) {
  type <- match.arg(type)
```

```
  out <- crr_american_with_nodes(S0, K, r, q, sigma_ann, T, Nsteps, type,
keep steps = c(1,2))
  if (!is.finite(out$price)) return(list(delta = NA, gamma = NA, theta =
NA, vega = NA, rho_1pp = NA))

  dt <- out$dt
  n1 <- out$nodes[["step 1"]]
  n2 <- out$nodes[["step_2"]]

  Vd <- n1$Valor_americano[n1$j == 0]
  Vu <- n1$Valor_americano[n1$j == 1]
  Sd <- n1$S[n1$j == 0]
  Su <- n1$S[n1$j == 1]
  delta <- (Vu - Vd) / (Su - Sd)

  Vdd <- n2$Valor_americano[n2$j == 0]
  Vud <- n2$Valor americano[n2$j == 1]
  Vuu <- n2$Valor_americano[n2$j == 2]

  Sdd <- n2$S[n2$j == 0]
  Sud <- n2$S[n2$j == 1]
  Suu <- n2$S[n2$j == 2]

  delta_up   <- (Vuu - Vud) / (Suu - Sud)
  delta_down <- (Vud - Vdd) / (Sud - Sdd)
  gamma <- (delta_up - delta_down) / ((Suu - Sdd) / 2)

  V0 <- out$price
  theta_per_year <- (Vud - V0) / (2 * dt)
  theta_per_day <- theta_per_year / 365

  price_up <- crr_price(S0, K, r, q, sigma_ann + bump_sigma_pto, T, Nsteps,
type = type, american = TRUE)
  price_dn <- crr_price(S0, K, r, q, sigma_ann - bump_sigma_pto, T, Nsteps,
type = type, american = TRUE)
  vega <- (price up - price dn) / (2 * bump sigma pto)

  price_ru <- crr_price(S0, K, r + bump_r_pp, q, sigma_ann, T, Nsteps, type
= type, american = TRUE)
  price rd <- crr price(S0, K, r - bump r pp, q, sigma ann, T, Nsteps, type
= type, american = TRUE)
  rho_1pp <- (price_ru - price_rd) / 2

  list(delta = delta, gamma = gamma, theta = theta per day, vega = vega,
rho 1pp = rho 1pp)
}

tabla_greeks <- do.call(rbind, lapply(1:nrow(pos), function(i) {
  tk <- pos$Accion[i]
  tipo <- pos$Tipo[i]
  K <- pos$K[i]
  contratos <- pos$Contratos[i]

  sigma use <- vol impl tv[tk]

  g <- crr_greeks(S0 = S0[tk], K = K, r = r, q = q_yield[tk],
                  sigma_ann = sigma_use, T = T, Nsteps = Nsteps, type =
tipo)
```

```
  data.frame(
    Accion = tk,
    Tipo = tipo,
    K = K,
    Sigma_usada_pct = sigma_use,
    Contratos = contratos,
    Delta_opcion = g$delta,
    Gamma opcion = g$gamma,
    Theta_opcion_USD_dia = g$theta,
    Vega_opcion_USD_por_1pto = g$vega,
    Rho_opcion_USD_por_1pp = g$rho_1pp,
    Delta_posicion_acciones = g$delta * contratos * mult,
    Gamma posicion = g$gamma * contratos * mult,
    Theta_posicion_USD_dia = g$theta * contratos * mult,
    Vega_posicion_USD_por_1pto = g$vega * contratos * mult,
    Rho_posicion_USD_por_1pp = g$rho_1pp * contratos * mult,
    row.names = NULL
  )
}))

print(tabla_greeks)

for (i in 1:nrow(pos)) {
  tk <- pos$Accion[i]
  tipo <- pos$Tipo[i]
  K <- pos$K[i]
  sigma_use <- vol_impl_tv[tk]

  cat("\n========================\n")
  cat("Nodos y decisión:", tk, tipo, "K =", K, "sigma =", sigma_use, "%\n")
  cat("========================\n")

  dbg <- crr_american_with_nodes(S0 = S0[tk], K = K, r = r, q =
q_yield[tk],
                                 sigma_ann = sigma_use, T = T, Nsteps =
Nsteps,
                                 type = tipo, keep steps = c(1,2,4))

  print(dbg$nodes[["step_1"]])
  print(dbg$nodes[["step_2"]])
  print(dbg$nodes[["step 4"]])
}
```

Bloque 7. Señal de volatilidad, straddle y rentabilidad a 2 semanas (revaluación con árbol)

```
tickers <- c("NVDA", "AMD", "XOM")

T total days nat <- 49
T total <- T total days nat / 365

T_2w_bd <- 10
T_2w_nat_equiv <- 14
```

```
T_rem <- (T_total_days_nat - T_2w_nat_equiv) / 365

# Ajuste (3): reducir Npaths por tiempo de cómputo
Npaths <- 5000  # se reduce para disminuir tiempo de ejecución
set.seed(123)
dt_year <- 1/252

S0_all <- as.numeric(datos[nrow(datos), tickers])
names(S0_all) <- tickers

razon_vol <- vol_garch_prom_34d[tickers] / vol_impl_tv[tickers]
senal_vol <- ifelse(is.na(razon_vol), NA_character_,
                    ifelse(razon_vol >= 1.10, "POSIBLE BARATA en vol (GARCH
> implícita)",
                    ifelse(razon_vol <= 0.90, "POSIBLE CARA en vol (GARCH <
implícita)",
                                              "SIN VENTAJA clara (|diff| <
10%)")))

tabla_senal <- data.frame(
  Accion = tickers,
  Sigma_GARCH_34d = as.numeric(vol_garch_prom_34d[tickers]),
  Sigma_implicita_TV = as.numeric(vol_impl_tv[tickers]),
  Razon = as.numeric(razon_vol),
  Senal_10pct = as.character(senal_vol),
  row.names = NULL
)
print(tabla_senal)

K_map <- c(NVDA = 200, AMD = 500, XOM = 185)

bid_call <- c(NVDA = NA_real_, AMD = NA_real_, XOM = NA_real_)
ask_call <- c(NVDA = NA_real_, AMD = NA_real_, XOM = NA_real_)
bid_put  <- c(NVDA = NA_real_, AMD = NA_real_, XOM = NA_real_)
ask_put  <- c(NVDA = NA_real_, AMD = NA_real_, XOM = NA_real_)

costo straddle <- ask call + ask put
eq_inf <- S0_all - costo_straddle
eq_sup <- S0_all + costo_straddle

tabla straddle <- data.frame(
  Accion = tickers,
  K = as.numeric(K_map[tickers]),
  Spot_hoy = as.numeric(S0_all[tickers]),
  Costo straddle ask = as.numeric(costo straddle[tickers]),
  Equilibrio inferior = as.numeric(eq inf[tickers]),
  Equilibrio_superior = as.numeric(eq_sup[tickers]),
  row.names = NULL
)
print(tabla straddle)

simula_S10_garch_Q <- function(S0, x_ret_pct, fit, N, T_var_bd, r, q,
dt_year) {
  x_ret_pct <- as.numeric(x_ret_pct)
  x ret pct <- x ret pct[is.finite(x ret pct)]

  omega <- fit$omega
  alpha <- fit$alpha
  beta  <- fit$beta
```

```
  tmp <- garch_last_h_and_next(x_ret_pct, fit)
  h_pct <- rep(tmp$h_next, N)

  S <- rep(S0, N)

  for (t in 1:T_var_bd) {
    z <- rnorm(N)

    e_pct <- sqrt(h_pct) * z
    h_dec <- h_pct / 10000
    shock_dec <- e_pct / 100

    drift <- (r - q) * dt_year - 0.5 * h_dec
    r_dec <- drift + shock_dec

    S <- S * exp(r_dec)

    h_pct <- omega + alpha * (e_pct^2) + beta * h_pct
  }

  S
}

Nsteps_reval <- 500

revalua_opcion_arbol <- function(S_now, K, tipo, sigma_ann, r, q, T_rem,
Nsteps) {
  crr_price(S0 = S_now, K = K, r = r, q = q, sigma_ann = sigma_ann,
            T = T_rem, Nsteps = Nsteps, type = tipo, american = TRUE)
}

pos <- data.frame(
  Accion = c("NVDA", "AMD", "XOM"),
  Tipo = c("call", "put", "call"),
  K = c(200, 500, 185),
  Contratos = c(2, 1, 3),
  stringsAsFactors = FALSE
)
mult <- 100

ask_pos <- c(NVDA = NA_real_, AMD = NA_real_, XOM = NA_real_)

res_2w <- lapply(1:nrow(pos), function(i) {
  tk <- pos$Accion[i]
  tipo <- pos$Tipo[i]
  K <- pos$K[i]
  contratos <- pos$Contratos[i]

  S10 <- simula_S10_garch_Q(
    S0 = S0_all[tk],
    x_ret_pct = ret[[tk]],
    fit = ajustes[[tk]],
    N = Npaths,
    T_var_bd = T_2w_bd,
    r = r,
    q = q_yield[tk],
    dt_year = dt_year
  )

  sigma_use <- vol_impl_tv[tk]
```

```
  q_use <- q_yield[tk]

  V10 <- vapply(S10, function(s) {
    revalua_opcion_arbol(S_now = s, K = K, tipo = tipo, sigma_ann =
sigma_use,
                         r = r, q = q_use, T_rem = T_rem, Nsteps =
Nsteps_reval)
  }, numeric(1))

  precio_pagado <- ask_pos[tk]
  pnl_por_accion <- V10 - precio_pagado
  pnl_pos <- pnl_por_accion * contratos * mult

  data.frame(
    Accion = tk,
    Tipo = tipo,
    K = K,
    Contratos = contratos,
    T_rem_anios = T_rem,
    Costo_total_USD = precio_pagado * contratos * mult,
    PnL_esperado_2sem_USD = mean(pnl_pos, na.rm = TRUE),
    Prob_de_ganar = mean(pnl_pos > 0, na.rm = TRUE),
    P5_USD = as.numeric(quantile(pnl_pos, probs = 0.05, na.rm = TRUE)),
    P95_USD = as.numeric(quantile(pnl_pos, probs = 0.95, na.rm = TRUE)),
    row.names = NULL
  )
})

tabla_2w <- do.call(rbind, res_2w)
print(tabla_2w)
```

[DONE]

2/11/2026 - 8:05:22 p.m.

Gracias por tu ayuda.



ChatGPT (GPT 5.2) •2/11/2026 - 8:05:23 p.m.

Con gusto. Si quieren, como siguiente paso puedo ayudarlos a dejar el Rmd “listo para entrega” con:

1. una sección de inputs al inicio (S0, r, q, strikes, bid y ask, IV de TradingView),
2. tablas formateadas y consistentes en unidades (porcentaje vs decimal),
3. un apéndice con supuestos y validaciones rápidas (convergencia del GARCH, p en CRR, etc.).

[DONE]

2/11/2026 - 8:05:40 p.m.

Por favor

ChatGPT (GPT 5.2) •2/11/2026 - 8:05:40 p.m.

¿Qué necesitas exactamente, por favor?

Puedo ayudarte con cualquiera de estas opciones:

1. Generar el archivo **Rmd completo** final (Bloques 1 a 7) ya listo para copiar y pegar.
2. Ajustar el Rmd a tu **CSV real** (nombres de columnas, formato de fecha, precios ajustados).
3. Armar una **versión para entrega** con tablas y conclusiones en texto (sin cambiar cálculos).
4. Revisar que todo sea consistente con sus supuestos: **T = 49/365**, simulación **34 días hábiles**, revaluación **(49-14)/365**, r y q.

Dime cuál de las cuatro quieres y si tu archivo se llama `precios_clase6.csv` o tiene otro nombre.

[DONE]

2/11/2026 - 8:05:54 p.m.

1

ChatGPT (GPT 5.2) •2/11/2026 - 8:05:54 p.m.

```
---
title: "Tarea Clase 6: GARCH, Monte Carlo y Árboles Binomiales (NVDA, AMD, XOM)"
author: "Equipo"
date: "`r format(Sys.Date())`"
output: html_document
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE, warning = FALSE, message = FALSE)
```
```

Bloque 1. Lectura de datos, rendimientos y volatilidades (30 días y 1 año)

```r
## R base

# Leer CSV (ajusta el nombre del archivo si aplica)
datos <- read.csv("precios_clase6.csv", stringsAsFactors = FALSE)

# Convertir y ordenar por fecha
datos$Date <- as.Date(datos$Date)
datos <- datos[order(datos$Date), ]

# Extraer precios
precios <- datos[, c("NVDA", "AMD", "XOM")]
precios <- as.data.frame(lapply(precios, as.numeric))

# Rendimientos logarítmicos diarios en porcentaje
ret <- as.data.frame(lapply(precios, function(p) 100 * diff(log(p))))
ret$Date <- datos$Date[-1]

# Volatilidad anualizada (252 días hábiles)
vol anual <- function(x) sd(x, na.rm = TRUE) * sqrt(252)

n <- nrow(ret)
ult30 <- max(1, n - 30 + 1):n
ulty  <- max(1, n - 252 + 1):n

vol30 <- sapply(ret[, c("NVDA","AMD","XOM")], function(x)
vol_anual(x[ult30]))
vol1y <- sapply(ret[, c("NVDA","AMD","XOM")], function(x)
vol anual(x[ulty]))

tabla_vol <- data.frame(
  Accion = c("NVDA", "AMD", "XOM"),
  Vol_30d_anual = as.numeric(vol30),
  Vol 1y anual  = as.numeric(vol1y),
  row.names = NULL
)

print(tabla_vol)
```

Bloque 2. Estimación GARCH(1,1) por MLE (innovaciones normales)

```r
## R base, optim con cotas y penalización para alpha+beta<1

fit_garch11_norm <- function(x) {
  x <- as.numeric(x)
  x <- x[is.finite(x)]
```

```r
  mu <- mean(x)
  e  <- x - mu
  n  <- length(e)

  h0 <- var(e)
  if (!is.finite(h0) || h0 <= 0) h0 <- 1

  nll <- function(par) {
    omega <- par[1]
    alpha <- par[2]
    beta  <- par[3]

    if (omega <= 0 || alpha < 0 || beta < 0) return(1e12)
    if (alpha + beta >= 0.999999) return(1e12 + 1e8 * (alpha + beta -
0.999999)^2)

    h <- numeric(n)
    h[1] <- h0
    for (t in 2:n) {
      h[t] <- omega + alpha * e[t - 1]^2 + beta * h[t - 1]
      if (!is.finite(h[t]) || h[t] <= 0) return(1e12)
    }

    ll <- -0.5 * (log(2 * pi) + log(h) + (e^2) / h)
    -sum(ll)
  }

  alpha0 <- 0.05
  beta0  <- 0.90
  omega0 <- max(1e-6, h0 * (1 - alpha0 - beta0))

  opt <- optim(
    par    = c(omega0, alpha0, beta0),
    fn     = nll,
    method = "L-BFGS-B",
    lower  = c(1e-12, 0, 0),
    upper  = c(Inf, 1, 1)
  )

  par <- opt$par
  omega <- par[1]; alpha <- par[2]; beta <- par[3]
  pers  <- alpha + beta

  # Reconstrucción de h_t para "volatilidad de hoy"
  h <- numeric(n)
  h[1] <- h0
  for (t in 2:n) h[t] <- omega + alpha * e[t - 1]^2 + beta * h[t - 1]

  h_lr <- omega / (1 - pers)
  vol_lr_ann    <- sqrt(h_lr) * sqrt(252)
  vol_today_ann <- sqrt(h[n]) * sqrt(252)

  half_life <- if (pers > 0 && pers < 1) log(0.5) / log(pers) else NA_real_

  list(
    omega = omega, alpha = alpha, beta = beta,
    persistencia = pers,
    vol_lr_ann = vol_lr_ann,
    vol_today_ann = vol_today_ann,
    half_life = half_life,
```

```
    converged = (opt$convergence == 0)
  )
}

tickers <- c("NVDA", "AMD", "XOM")
ajustes <- lapply(tickers, function(tk) fit_garch11_norm(ret[[tk]]))
names(ajustes) <- tickers

tabla_garch <- data.frame(
  Accion = tickers,
  omega = sapply(ajustes, `[[`, "omega"),
  alpha = sapply(ajustes, `[[`, "alpha"),
  beta  = sapply(ajustes, `[[`, "beta"),
  Persistencia = sapply(ajustes, `[[`, "persistencia"),
  Vol_largo_plazo_anual = sapply(ajustes, `[[`, "vol_lr_ann"),
  Vol_hoy_anual = sapply(ajustes, `[[`, "vol_today_ann"),
  Vida_media_shock_dias = sapply(ajustes, `[[`, "half_life"),
  Convergio = sapply(ajustes, `[[`, "converged"),
  row.names = NULL
)

print(tabla_garch)
```

Bloque 3. EWMA y pronóstico GARCH por reversión a la media (y razón vs IV)

```
tickers <- c("NVDA", "AMD", "XOM")
lambda <- 0.94

# EWMA de hoy (anualizada) en % usando rendimientos en % diario
ewma vol hoy <- function(x, lambda = 0.94) {
  x <- as.numeric(x)
  x <- x[is.finite(x)]
  n <- length(x)
  if (n < 2) return(NA_real_)

  e <- x - mean(x)
  s2 <- var(e)
  if (!is.finite(s2) || s2 <= 0) s2 <- 1

  for (t in 2:n) s2 <- lambda * s2 + (1 - lambda) * e[t - 1]^2
  sqrt(s2) * sqrt(252)
}

# Histórica 1 año: últimos 252 rendimientos, anualizada, en %
vol_hist_1y <- function(x) {
  x <- as.numeric(x)
  x <- x[is.finite(x)]
  n <- length(x)
  idx <- max(1, n - 252 + 1):n
  sd(x[idx], na.rm = TRUE) * sqrt(252)
}

vol_ewma_hoy <- sapply(tickers, function(tk) ewma_vol_hoy(ret[[tk]],
lambda))
vol_hist_1y_vec <- sapply(tickers, function(tk) vol_hist_1y(ret[[tk]]))
```

```r
# h_t (última) y h_{t+1} (día siguiente) para pronóstico y simulación
garch_last_h_and_next <- function(x, fit) {
  x <- as.numeric(x)
  x <- x[is.finite(x)]
  mu <- mean(x)
  e  <- x - mu
  n  <- length(e)

  omega <- fit$omega
  alpha <- fit$alpha
  beta  <- fit$beta

  h0 <- var(e)
  if (!is.finite(h0) || h0 <= 0) h0 <- 1

  h <- numeric(n)
  h[1] <- h0
  for (t in 2:n) h[t] <- omega + alpha * e[t - 1]^2 + beta * h[t - 1]

  h_last <- h[n]
  e_last <- e[n]
  h_next <- omega + alpha * e_last^2 + beta * h_last

  list(h_last = h_last, h_next = h_next)
}

# Pronóstico por reversión a la media arrancando en h_{t+1}
garch_forecast_var_from_next <- function(x, fit, horizons = c(10, 30, 34,
126, 252)) {
  tmp <- garch_last_h_and_next(x, fit)

  omega <- fit$omega
  alpha <- fit$alpha
  beta  <- fit$beta
  pers  <- alpha + beta

  h_bar <- omega / (1 - pers)
  h1 <- tmp$h_next

  fvar <- sapply(horizons, function(H) {
    if (H <= 1) return(h1)
    h_bar + (pers^(H - 1)) * (h1 - h_bar)
  })

  list(h1 = h1, h bar = h bar, pers = pers, fvar = fvar)
}

horizons <- c(10, 30, 34, 126, 252)

vol garch h <- matrix(NA real , nrow = length(tickers), ncol =
length(horizons),
                      dimnames = list(tickers, paste0(horizons, "d")))
vol_garch_prom_34d <- setNames(rep(NA_real_, length(tickers)), tickers)

for (tk in tickers) {
  fc <- garch_forecast_var_from_next(ret[[tk]], ajustes[[tk]], horizons)

  vol_garch_h[tk, ] <- sqrt(fc$fvar) * sqrt(252)
```

```
  # Volatilidad promedio esperada hasta vencimiento usando 34 días hábiles
  Tpath <- 34
  k <- 1:Tpath
  fvar_path <- ifelse(k == 1, fc$h1, fc$h_bar + (fc$pers^(k - 1)) * (fc$h1
- fc$h_bar))
  vol_garch_prom_34d[tk] <- sqrt(mean(fvar_path)) * sqrt(252)
}

# Volatilidad implícita TradingView (llenar, % anual)
vol_impl_tv <- c(NVDA = NA_real_, AMD = NA_real_, XOM = NA_real_)

razon <- vol_garch_prom_34d / vol_impl_tv
senal <- ifelse(is.na(razon), NA_character_,
                ifelse(razon >= 1.10, "GARCH > implícita (posible barata en
vol)",
                ifelse(razon <= 0.90, "GARCH < implícita (posible cara en
vol)",
                                      "Sin ventaja clara (|diff| < 10%)")))

tabla_vols <- data.frame(
  Accion = tickers,
  Vol_hist_1y_anual = as.numeric(vol_hist_1y_vec),
  Vol_EWMA_hoy_anual = as.numeric(vol_ewma_hoy),
  Vol_prom_GARCH_34d = as.numeric(vol_garch_prom_34d),
  Sigma_implicita_TV = as.numeric(vol_impl_tv),
  Razon_GARCH_sobre_Impl = as.numeric(razon),
  Senal_10pct = as.character(senal),
  row.names = NULL
)

tabla_garch_plazos <- data.frame(
  Accion = tickers,
  Vol_GARCH_10d  = as.numeric(vol_garch_h[, "10d"]),
  Vol_GARCH_30d  = as.numeric(vol_garch_h[, "30d"]),
  Vol_GARCH_34d  = as.numeric(vol_garch_h[, "34d"]),
  Vol_GARCH_126d = as.numeric(vol_garch_h[, "126d"]),
  Vol_GARCH_252d = as.numeric(vol_garch_h[, "252d"]),
  row.names = NULL
)

print(tabla_vols)
print(tabla_garch_plazos)
```

Bloque 4. Monte Carlo con GARCH bajo medida neutral al riesgo (34 días hábiles)

```
set.seed(123)
N <- 100000
T_venc_bd <- 34
T_var_bd  <- 10
dt_year <- 1/252

tickers <- c("NVDA", "AMD", "XOM")
```

```r
S0 <- as.numeric(datos[nrow(datos), tickers])
names(S0) <- tickers

q_yield <- c(NVDA = 0.0047, AMD = 0.0000, XOM = 0.0225)
r <- 0.0394

# Simulación bajo Q con corrección de Itô y varianza inicial h {t+1}
simula_precio_garch_Q <- function(S0, x_ret_pct, fit, N, T_venc_bd,
T_var_bd, r, q, dt_year) {
  x_ret_pct <- as.numeric(x_ret_pct)
  x_ret_pct <- x_ret_pct[is.finite(x_ret_pct)]

  omega <- fit$omega
  alpha <- fit$alpha
  beta  <- fit$beta

  tmp <- garch last h and next(x ret pct, fit)
  h_pct <- rep(tmp$h_next, N)

  S <- rep(S0, N)
  S10 <- rep(NA_real_, N)

  for (t in 1:T_venc_bd) {
    z <- rnorm(N)

    e_pct <- sqrt(h_pct) * z
    h_dec <- h_pct / 10000
    shock_dec <- e_pct / 100

    drift <- (r - q) * dt_year - 0.5 * h_dec
    r_dec <- drift + shock_dec

    S <- S * exp(r_dec)

    h pct <- omega + alpha * (e pct^2) + beta * h pct

    if (t == T_var_bd) S10 <- S
  }

  p <- as.numeric(quantile(S, probs = c(0.05, 0.25, 0.50, 0.75, 0.95),
na.rm = TRUE))
  names(p) <- c("P5","P25","P50","P75","P95")

  prob subir <- mean(S > S0)

  loss10 <- (S0 - S10) / S0
  var99_10d <- as.numeric(quantile(loss10, probs = 0.99, na.rm = TRUE))

  list(P5 = p["P5"], P25 = p["P25"], P50 = p["P50"], P75 = p["P75"], P95 =
p["P95"],
       prob_subir = prob_subir, var99_10d = var99_10d)
}

res <- lapply(tickers, function(tk) {
  simula precio garch Q(
    S0 = S0[tk],
    x_ret_pct = ret[[tk]],
    fit = ajustes[[tk]],
    N = N,
```

```
    T_venc_bd = T_venc_bd,
    T_var_bd = T_var_bd,
    r = r,
    q = q_yield[tk],
    dt_year = dt_year
  )
})
names(res) <- tickers

tabla_mc <- data.frame(
  Accion = tickers,
  Precio_hoy = as.numeric(S0[tickers]),
  P5  = sapply(res, `[[`, "P5"),
  P25 = sapply(res, `[[`, "P25"),
  P50 = sapply(res, `[[`, "P50"),
  P75 = sapply(res, `[[`, "P75"),
  P95 = sapply(res, `[[`, "P95"),
  Prob de subir = sapply(res, `[[`, "prob subir"),
  VaR_99pct_10d = sapply(res, `[[`, "var99_10d"),
  row.names = NULL
)

print(tabla_mc)
```

Bloque 5. Árbol binomial CRR americano (500 pasos), T = 49/365, q continuo

```
r <- 0.0394
T <- 49/365
Nsteps <- 500

q_yield <- c(NVDA = 0.0047, AMD = 0.0000, XOM = 0.0225)

inst <- data.frame(
  Accion = c("NVDA", "AMD", "XOM"),
  Tipo   = c("call", "put", "call"),
  K      = c(200, 500, 185),
  stringsAsFactors = FALSE
)

S0 <- as.numeric(datos[nrow(datos), inst$Accion])
names(S0) <- inst$Accion

# Ajuste final (1): renombrar vector para no pisar la función vol_hist_1y
del Bloque 3
vol_hist_1y_vec <- sapply(inst$Accion, function(tk) vol_hist_1y(ret[[tk]]))

vol_garch_34 <- vol_garch_prom_34d[inst$Accion]

# Entradas a llenar
vol_impl_tv <- c(NVDA = NA_real_, AMD = NA_real_, XOM = NA_real_)
bid_mkt <- c(NVDA = NA_real_, AMD = NA_real_, XOM = NA_real_)
ask_mkt <- c(NVDA = NA_real_, AMD = NA_real_, XOM = NA_real_)
```

```
crr_price <- function(S0, K, r, q, sigma_ann, T, Nsteps, type =
c("call","put"), american = TRUE) {
  type <- match.arg(type)

  sigma <- sigma_ann / 100
  dt <- T / Nsteps
  u <- exp(sigma * sqrt(dt))
  d <- 1 / u

  disc <- exp(-r * dt)
  p <- (exp((r - q) * dt) - d) / (u - d)
  if (!is.finite(p) || p < 0 || p > 1) return(NA real )

  j <- 0:Nsteps
  ST <- S0 * (u^j) * (d^(Nsteps - j))
  if (type == "call") V <- pmax(ST - K, 0) else V <- pmax(K - ST, 0)

  for (i in (Nsteps - 1):0) {
    V <- disc * (p * V[2:(i + 2)] + (1 - p) * V[1:(i + 1)])

    if (american) {
      j <- 0:i
      Sij <- S0 * (u^j) * (d^(i - j))
      if (type == "call") intrinsic <- pmax(Sij - K, 0) else intrinsic <-
pmax(K - Sij, 0)
      V <- pmax(V, intrinsic)
    }
  }

  V[1]
}

valua_instrumento <- function(tk, tipo, K, r, q, T, Nsteps,
                              sigma_hist, sigma_garch, sigma_impl, bid,
ask) {
  sigmas <- c(Historica 1y = sigma hist, GARCH 34d = sigma garch,
Implicita_TV = sigma_impl)

  out <- lapply(names(sigmas), function(nm) {
    sig <- sigmas[[nm]]

    price_eur <- crr_price(S0 = S0[tk], K = K, r = r, q = q, sigma_ann =
sig,
                           T = T, Nsteps = Nsteps, type = tipo, american =
FALSE)
    price_ame <- crr_price(S0 = S0[tk], K = K, r = r, q = q, sigma_ann =
sig,
                           T = T, Nsteps = Nsteps, type = tipo, american =
TRUE)

    data.frame(
      Accion = tk,
      Tipo = tipo,
      K = K,
      # Ajuste final (2): quitar sufijos de nombre con unname()
      Sigma_fuente = unname(nm),
      Sigma_anual_pct = sig,
      Precio_Europeo = price_eur,
      Precio_Americano = price_ame,
```

```
      Prima_ejercicio_anticipado = price_ame - price_eur,
      Bid = bid,
      Ask = ask,
      row.names = NULL,
      stringsAsFactors = FALSE
    )
  })

  do.call(rbind, out)
}

tabla_arbol <- do.call(rbind, lapply(1:nrow(inst), function(i) {
  tk <- inst$Accion[i]
  valua_instrumento(
    tk = tk,
    tipo = inst$Tipo[i],
    K = inst$K[i],
    r = r,
    q = q_yield[tk],
    T = T,
    Nsteps = Nsteps,
    sigma_hist = vol_hist_1y_vec[tk],
    sigma_garch = vol_garch_34[tk],
    sigma_impl = vol_impl_tv[tk],
    bid = bid_mkt[tk],
    ask = ask_mkt[tk]
  )
}))

print(tabla_arbol)
```

Bloque 6. Griegas (delta, gamma, theta) y vega, rho; nodos con decisión

```
pos <- data.frame(
  Accion = c("NVDA", "AMD", "XOM"),
  Tipo   = c("call", "put", "call"),
  K      = c(200, 500, 185),
  Contratos = c(2, 1, 3),
  stringsAsFactors = FALSE
)
mult <- 100

bump_sigma_pto <- 1.0
bump_r_pp <- 0.01

crr_american_with_nodes <- function(S0, K, r, q, sigma_ann, T, Nsteps, type
= c("call","put"),
                                    keep_steps = c(1,2,4)) {
  type <- match.arg(type)

  sigma <- sigma_ann / 100
  dt <- T / Nsteps
  u <- exp(sigma * sqrt(dt))
  d <- 1 / u
  disc <- exp(-r * dt)
```

```
  p <- (exp((r - q) * dt) - d) / (u - d)

  if (!is.finite(p) || p < 0 || p > 1) return(list(price = NA real , nodes
= NULL, dt = dt))

  j <- 0:Nsteps
  ST <- S0 * (u^j) * (d^(Nsteps - j))
  if (type == "call") V <- pmax(ST - K, 0) else V <- pmax(K - ST, 0)

  nodes <- list()

  for (i in (Nsteps - 1):0) {
    V cont <- disc * (p * V[2:(i + 2)] + (1 - p) * V[1:(i + 1)])

    j <- 0:i
    Sij <- S0 * (u^j) * (d^(i - j))
    if (type == "call") intrinsic <- pmax(Sij - K, 0) else intrinsic <-
pmax(K - Sij, 0)

    exercise <- intrinsic > V_cont
    V <- pmax(V_cont, intrinsic)

    if (i %in% keep_steps) {
      nodes[[paste0("step_", i)]] <- data.frame(
        step = i,
        j = j,
        S = Sij,
        Continuar = V_cont,
        Intrinseco = intrinsic,
        Valor_americano = V,
        Decision = ifelse(exercise, "EJERCER", "ESPERAR"),
        row.names = NULL
      )
    }
  }

  list(price = V[1], nodes = nodes, dt = dt)
}

crr_greeks <- function(S0, K, r, q, sigma_ann, T, Nsteps, type =
c("call","put")) {
  type <- match.arg(type)

  out <- crr_american_with_nodes(S0, K, r, q, sigma_ann, T, Nsteps, type,
keep steps = c(1,2))
  if (!is.finite(out$price)) return(list(delta = NA, gamma = NA, theta =
NA, vega = NA, rho_1pp = NA))

  dt <- out$dt
  n1 <- out$nodes[["step 1"]]
  n2 <- out$nodes[["step 2"]]

  Vd <- n1$Valor_americano[n1$j == 0]
  Vu <- n1$Valor_americano[n1$j == 1]
  Sd <- n1$S[n1$j == 0]
  Su <- n1$S[n1$j == 1]
  delta <- (Vu - Vd) / (Su - Sd)

  Vdd <- n2$Valor_americano[n2$j == 0]
  Vud <- n2$Valor_americano[n2$j == 1]
```

```
  Vuu <- n2$Valor_americano[n2$j == 2]

  Sdd <- n2$S[n2$j == 0]
  Sud <- n2$S[n2$j == 1]
  Suu <- n2$S[n2$j == 2]

  delta_up   <- (Vuu - Vud) / (Suu - Sud)
  delta down <- (Vud - Vdd) / (Sud - Sdd)
  gamma <- (delta_up - delta_down) / ((Suu - Sdd) / 2)

  V0 <- out$price
  theta_per_year <- (Vud - V0) / (2 * dt)
  theta per day <- theta per year / 365

  price_up <- crr_price(S0, K, r, q, sigma_ann + bump_sigma_pto, T, Nsteps,
type = type, american = TRUE)
  price_dn <- crr_price(S0, K, r, q, sigma_ann - bump_sigma_pto, T, Nsteps,
type = type, american = TRUE)
  vega <- (price_up - price_dn) / (2 * bump_sigma_pto)

  price_ru <- crr_price(S0, K, r + bump_r_pp, q, sigma_ann, T, Nsteps, type
= type, american = TRUE)
  price_rd <- crr_price(S0, K, r - bump_r_pp, q, sigma_ann, T, Nsteps, type
= type, american = TRUE)
  rho_1pp <- (price_ru - price_rd) / 2

  list(delta = delta, gamma = gamma, theta = theta_per_day, vega = vega,
rho_1pp = rho_1pp)
}

tabla_greeks <- do.call(rbind, lapply(1:nrow(pos), function(i) {
  tk <- pos$Accion[i]
  tipo <- pos$Tipo[i]
  K <- pos$K[i]
  contratos <- pos$Contratos[i]

  sigma use <- vol impl tv[tk]

  g <- crr_greeks(S0 = S0[tk], K = K, r = r, q = q_yield[tk],
                  sigma_ann = sigma_use, T = T, Nsteps = Nsteps, type =
tipo)

  data.frame(
    Accion = tk,
    Tipo = tipo,
    K = K,
    Sigma_usada_pct = sigma_use,
    Contratos = contratos,
    Delta_opcion = g$delta,
    Gamma opcion = g$gamma,
    Theta opcion USD dia = g$theta,
    Vega_opcion_USD_por_1pto = g$vega,
    Rho_opcion_USD_por_1pp = g$rho_1pp,
    Delta_posicion_acciones = g$delta * contratos * mult,
    Gamma posicion = g$gamma * contratos * mult,
    Theta posicion USD dia = g$theta * contratos * mult,
    Vega_posicion_USD_por_1pto = g$vega * contratos * mult,
    Rho_posicion_USD_por_1pp = g$rho_1pp * contratos * mult,
    row.names = NULL
  )
```

```
}))

print(tabla_greeks)

for (i in 1:nrow(pos)) {
  tk <- pos$Accion[i]
  tipo <- pos$Tipo[i]
  K <- pos$K[i]
  sigma_use <- vol_impl_tv[tk]

  cat("\n========================\n")
  cat("Nodos y decisión:", tk, tipo, "K =", K, "sigma =", sigma_use, "%\n")
  cat("========================\n")

  dbg <- crr_american_with_nodes(S0 = S0[tk], K = K, r = r, q =
q_yield[tk],
                                 sigma_ann = sigma_use, T = T, Nsteps =
Nsteps,
                                 type = tipo, keep_steps = c(1,2,4))

  print(dbg$nodes[["step_1"]])
  print(dbg$nodes[["step_2"]])
  print(dbg$nodes[["step_4"]])
}
```

Bloque 7. Señal, straddle y rentabilidad a 2 semanas (revaluación con árbol)

```
tickers <- c("NVDA", "AMD", "XOM")

T_total_days_nat <- 49
T_2w_bd <- 10
T_2w_nat_equiv <- 14
T_rem <- (T_total_days_nat - T_2w_nat_equiv) / 365

# Se reduce para disminuir tiempo de ejecución
Npaths <- 5000
set.seed(123)
dt_year <- 1/252

S0_all <- as.numeric(datos[nrow(datos), tickers])
names(S0_all) <- tickers

# Señal de volatilidad (umbral 10%)
razon_vol <- vol_garch_prom_34d[tickers] / vol_impl_tv[tickers]
senal_vol <- ifelse(is.na(razon_vol), NA_character_,
                    ifelse(razon_vol >= 1.10, "POSIBLE BARATA en vol (GARCH
> implícita)",
                    ifelse(razon_vol <= 0.90, "POSIBLE CARA en vol (GARCH <
implícita)",
                                              "SIN VENTAJA clara (|diff| <
10%)")))

tabla_senal <- data.frame(
  Accion = tickers,
  Sigma_GARCH_34d = as.numeric(vol_garch_prom_34d[tickers]),
```

```
  Sigma_implicita_TV = as.numeric(vol_impl_tv[tickers]),
  Razon = as.numeric(razon_vol),
  Senal 10pct = as.character(senal vol),
  row.names = NULL
)
print(tabla_senal)

# Straddle: costo y puntos de equilibrio (requiere bid y ask de call y put)
K_map <- c(NVDA = 200, AMD = 500, XOM = 185)

bid_call <- c(NVDA = NA_real_, AMD = NA_real_, XOM = NA_real_)
ask_call <- c(NVDA = NA_real_, AMD = NA_real_, XOM = NA_real_)
bid put  <- c(NVDA = NA real , AMD = NA real , XOM = NA real )
ask_put  <- c(NVDA = NA_real_, AMD = NA_real_, XOM = NA_real_)

costo_straddle <- ask_call + ask_put
eq_inf <- S0_all - costo_straddle
eq sup <- S0 all + costo straddle

tabla_straddle <- data.frame(
  Accion = tickers,
  K = as.numeric(K_map[tickers]),
  Spot_hoy = as.numeric(S0_all[tickers]),
  Costo_straddle_ask = as.numeric(costo_straddle[tickers]),
  Equilibrio_inferior = as.numeric(eq_inf[tickers]),
  Equilibrio_superior = as.numeric(eq_sup[tickers]),
  row.names = NULL
)
print(tabla_straddle)

# Simular S_10 con GARCH bajo Q (sin drift histórico)
simula_S10_garch_Q <- function(S0, x_ret_pct, fit, N, T_var_bd, r, q,
dt_year) {
  x_ret_pct <- as.numeric(x_ret_pct)
  x_ret_pct <- x_ret_pct[is.finite(x_ret_pct)]

  omega <- fit$omega
  alpha <- fit$alpha
  beta  <- fit$beta

  tmp <- garch last h and next(x ret pct, fit)
  h_pct <- rep(tmp$h_next, N)

  S <- rep(S0, N)

  for (t in 1:T var bd) {
    z <- rnorm(N)

    e_pct <- sqrt(h_pct) * z
    h dec <- h pct / 10000
    shock dec <- e pct / 100

    drift <- (r - q) * dt_year - 0.5 * h_dec
    r_dec <- drift + shock_dec

    S <- S * exp(r dec)

    h_pct <- omega + alpha * (e_pct^2) + beta * h_pct
  }
```

```
  S
}

# Revaluación a 10 días hábiles con CRR americano y tiempo remanente (49-
14)/365
Nsteps_reval <- 500

revalua opcion arbol <- function(S now, K, tipo, sigma ann, r, q, T rem,
Nsteps) {
  crr_price(S0 = S_now, K = K, r = r, q = q, sigma_ann = sigma_ann,
            T = T_rem, Nsteps = Nsteps, type = tipo, american = TRUE)
}

pos <- data.frame(
  Accion = c("NVDA", "AMD", "XOM"),
  Tipo = c("call", "put", "call"),
  K = c(200, 500, 185),
  Contratos = c(2, 1, 3),
  stringsAsFactors = FALSE
)
mult <- 100

# Ask pagado por cada posición (USD por acción)
ask_pos <- c(NVDA = NA_real_, AMD = NA_real_, XOM = NA_real_)

res_2w <- lapply(1:nrow(pos), function(i) {
  tk <- pos$Accion[i]
  tipo <- pos$Tipo[i]
  K <- pos$K[i]
  contratos <- pos$Contratos[i]

  S10 <- simula_S10_garch_Q(
    S0 = S0_all[tk],
    x_ret_pct = ret[[tk]],
    fit = ajustes[[tk]],
    N = Npaths,
    T var bd = T 2w bd,
    r = r,
    q = q_yield[tk],
    dt_year = dt_year
  )

  sigma_use <- vol_impl_tv[tk]
  q_use <- q_yield[tk]

  V10 <- vapply(S10, function(s) {
    revalua_opcion_arbol(S_now = s, K = K, tipo = tipo, sigma_ann =
sigma_use,
                         r = r, q = q_use, T_rem = T_rem, Nsteps =
Nsteps reval)
  }, numeric(1))

  precio_pagado <- ask_pos[tk]
  pnl_por_accion <- V10 - precio_pagado
  pnl pos <- pnl por accion * contratos * mult

  data.frame(
    Accion = tk,
    Tipo = tipo,
    K = K,
```

```
    Contratos = contratos,
    T_rem_anios = T_rem,
    Costo total USD = precio pagado * contratos * mult,
    PnL_esperado_2sem_USD = mean(pnl_pos, na.rm = TRUE),
    Prob_de_ganar = mean(pnl_pos > 0, na.rm = TRUE),
    P5_USD = as.numeric(quantile(pnl_pos, probs = 0.05, na.rm = TRUE)),
    P95_USD = as.numeric(quantile(pnl_pos, probs = 0.95, na.rm = TRUE)),
    row.names = NULL
  )
})

tabla_2w <- do.call(rbind, res_2w)
print(tabla_2w)
```

[DONE]

2/11/2026 - 8:08:55 p.m.

Gracias, lo tomo en cuenta.



ChatGPT (GPT 5.2) •2/11/2026 - 8:08:56 p.m.

Perfecto. Si después quieren dejarlo aún más sólido para entrega, puedo ayudarte a:

1. Añadir una sección inicial de “Inputs” donde solo cambian archivo, IV, bid y ask, strikes y contratos.
2. Formatear tablas con redondeo consistente y unidades explícitas (porcentaje vs decimal, USD por acción vs por contrato).
3. Agregar un apartado breve de validaciones: p del CRR dentro de [0,1], convergencia del GARCH, y chequeos de coherencia (MC vs árbol con σ GARCH).

[DONE]

2/11/2026 - 8:09:10 p.m.

No es necesario por el momento.



ChatGPT (GPT 5.2) •2/11/2026 - 8:09:10 p.m.

Entendido. Cuando lo necesites, dime qué parte quieren pulir primero (tablas, narrativa, supuestos, o validaciones) y lo armamos rápido.

[DONE]